

Multi-Line Insurance Policy and Claims Management System

1. Problem Statement

An insurance carrier, which manages vehicle, property, and life insurance, currently uses a legacy system that results in **manual, slow, and inconsistent policy quoting, issuance, and claims handling**. The lack of a unified platform makes cross-selling difficult, and the manual claims review process leads to long resolution times, high operational costs, and poor customer experience.

The business needs a single Salesforce solution to:

1. **Standardize and accelerate** the quoting and policy issuance process for all insurance lines.
2. **Automate claim initiation and routing** based on policy type and claim severity.
3. Provide a **360-degree view** of the Customer, Policy, and Claim data in one place.

2. Objectives

- **Design a flexible data model** capable of storing diverse product data (e.g., vehicle VIN for Vehicle, square footage for Property).
- **Implement Screen Flows** to guide agents through complex, multi-step quoting processes for different policy types.
- **Develop a dynamic Apex/LWC Claims Dashboard** for adjusters to efficiently manage their workload.
- **Automate claims approval/escalation** based on claim amount and policy history using Record-Triggered Flows.
- Enforce **data validation** across all insurance products to ensure compliance and accuracy.

3. Challenges

- **Variable Data Management:** Handling the diverse fields required for Vehicle, Property, and Life insurance within a unified object structure using Record Types/Field Sets.
- **Approval Complexity:** Designing a Claims Approval Flow that dynamically routes based on policy type, claim amount, and adjuster level.
- **API Integration Simulation (Apex):** Simulating a callout to an external motor vehicle or property database for quote verification (requires mock test classes).
- **User Adoption:** Creating intuitive Screen Flows that minimize clicks and maximize agent efficiency during the quoting process.

4. Solution Approach

The solution leverages **Screen Flows** for interactive quoting, a flexible **Custom Data Model** for policy diversity, and **LWC/Apex** for the high-performance Claims Management dashboard.

1. **Flexible Data Model:** Create **Policy** and **Claim** objects with different **Record Types** for Vehicle, Property, and Life.
2. **Quoting Flows (Declarative):** Build multi-screen Screen Flows to capture policy-specific data efficiently.

3. **Claims Logic (Programmatic/Flow):** Use Apex for complex rating logic and Record-Triggered Flow for automated routing.
4. **UI Development (LWC):** Create a highly focused Claims Management dashboard for adjusters.

5. Milestones Milestone 1: Core Data Model & Policy Configuration User Story1: Creation of custom objects

1. Create Policy custom object:

- Go to setup, Click on object Manager.
- Click on create and select custom object.
- Fill the following details:
 - Label: Policy
 - Plural Label: Policies
 - Object Name: Auto Populated • Record Name: Policy Number
 - Data type: Auto Number
 - Format: P - {0000} • Starting Number: 1 • Under optional features:
 - Allow Reports, Activities and Track field History • Under Deployment Status:
 - Select the deployed radio button.
 - Under Search Status:
 - Check the Allow Search checkbox.
 - Click on save.

The screenshot shows the Salesforce 'Policy' Custom Object Definition Edit page. The left sidebar contains a navigation menu with the following items: Details (selected), Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, and Search Layouts. The main content area is titled 'Policy' and includes a 'Custom Object Definition Edit' header with 'Save', 'Save & New', and 'Cancel' buttons. Below this is the 'Custom Object Information' section, which contains the following fields and options:

- Label:** Policy (Example: Account)
- Plural Label:** Policies (Example: Accounts)
- Starts with vowel sound:** ☐
- The Object Name is used when referencing the object via the API.**
 - Object Name:** Policy (Example: Account)
 - Description:** (Empty text area)
- Context-Sensitive Help Setting:**
 - ☒ Open the standard Salesforce.com Help & Training window
 - ☐ Open a window using a Visualforce page

Setup Home Object Manager

SETUP > OBJECT MANAGER

Policy

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts

Enter Record Name Label and Format

The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field is always called "Name" when referenced via the API.

Record Name Example: Account Name

Data Type Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

Display Format Example: A-{0000} [What is This?](#)

Optional Features

☒ Allow Reports
☒ Allow Activities
☒ Track Field History
☐ Allow in Chatter Groups
☐ Enable Licensing [i](#)

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more.](#)

☒ Allow Sharing
☒ Allow Bulk API Access
☒ Allow Streaming API Access

Deployment Status

[What is this?](#)

☐ In Development
☒ Deployed

Search Status

When this setting is enabled, your users can find records of this object type when they search. [Learn more.](#)

☒ Allow Search

2. Define Record Types on Policy Object:

- Go to setup, Click on Object Manager.
- Search for Policy objects and click on Policy__c.
- In the left side panel, click on the record types.
- Click on New.
- Fill out the following details:
 - Select the existing record type as Master.
 - Give record type label as "Auto"
 - Record Type Name will be auto populated.
 - Give a meaningful description.
 - Check the active checkbox.
 - Check the Make Available checkbox for all the profiles. ○
- Click on Next.
- It will redirect to the Assign page layouts page, Leave everything as is, and click on save.

SETUP > OBJECT MANAGER

Policy

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts

Edit Record Type

Auto

Enter a new name for the selected record type and click Save.

Record Type

Record Type Label:

Record Type Name:

Namespace Prefix:

Description:

Active: ☒

Save Cancel

Repeat the same steps for creating the property and life record types as well.

3. Create Field Sets for policy specific data:

- Go to setup, Click on Object Manager.
- Search for Policy object and click on Policy__c.
- In the left side panel, click on the Field Sets.
- Click on New.
- Fill out the following details:
 - Field Set Label as Vehicle
 - Field Set Name will be Auto populated.
 - Where is this used: In the policy object.
- Click on save.
- Click on the field set you have just created and it will redirect to the page layout setup.
- Drag VIN, and model year fields to the field set and click on save.

SETUP > OBJECT MANAGER

Policy

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts

Vehicle Fields

Save Cancel Undo Redo Field Set Properties

Quick Find Policy Name

Beneficiary Name	Deleted	Last Referenced Date	Policy Number	Premium	System Modstamp
Created By	Last Activity Date	Last Viewed Date	Policy Start Date	Record ID	VIN
Created Date	Last Modified By	Model Year	Policy State	Record Type	Year Built
Customer	Last Modified Date	Owner	Policy Term Months	Square Footage	

Drag any of the fields above into one of the lists below.

Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

VIN

Model Year

Repeat the same steps for creating the property and life Field sets as well.

3.1 Property FieldSet Requires:

- Square footage
- Year Built

Setup > OBJECT MANAGER > Policy

Property Fields

Save Cancel Undo Redo Field Set Properties

Quick Find Policy Name

Beneficiary Name	Deleted	Last Referenced Date	Policy Number	Premium	System Modstamp
Created By	Last Activity Date	Last Viewed Date	Policy Start Date	Record ID	VIN
Created Date	Last Modified By	Model Year	Policy State	Record Type	Year Built
Customer	Last Modified Date	Owner	Policy Term Months	Square Footage	

Drag any of the fields above into one of the lists below.

Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

Square Footage
Year Built

3.2 Life FieldSet Requires:

- Beneficiary Name
- Policy Term Months

Setup > OBJECT MANAGER > Policy

Life Fields

Save Cancel Undo Redo Field Set Properties

Quick Find Policy Name

Beneficiary Name	Deleted	Last Referenced Date	Policy Number	Premium	System Modstamp
Created By	Last Activity Date	Last Viewed Date	Policy Start Date	Record ID	VIN
Created Date	Last Modified By	Model Year	Policy State	Record Type	Year Built
Customer	Last Modified Date	Owner	Policy Term Months	Square Footage	

Drag any of the fields above into one of the lists below.

Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

Beneficiary Name
Policy Term Months

4. Create Fields in the Policy Object:

S.No	Name of the Field	Data Type	Required
1	Beneficiary Name	Text(50)	
2	Customer	Lookup(Contact)	
3	Model Year	Text(5)	

4	Policy Start Date	Date	
5	Policy Term Months	Number(18, 0)	
6	Premium	Currency(16, 2)	
7	Square Footage	Number(18, 0)	
8	VIN	Text(20)	
9	Year Built	Text(5)	

5. Create Claim custom object:

- Go to setup, Click on object Manager.
- Click on create and select custom object.
- Fill the following details:
- Label: Claim
- Plural Label: Claim
- Object Name: Auto Populated
- Record Name: Claim Number
- Data type: Auto Number
- Format: C - {0000}
- Starting Number: 1
- Under optional features:
- Allow Reports, Activities and Track field History
- Under Deployment Status:
- Select the deployed radio button.
- Under Search Status:
- Check the Allow Search checkbox.
- Click on save.

Setup Home Object Manager

SETUP > OBJECT MANAGER

Claim

Details

- Fields & Relationships
- Page Layouts
- Lightning Record Pages
- Buttons, Links, and Actions
- Compact Layouts
- Field Sets
- Object Limits
- Record Types
- Related Lookup Filters
- Search Layouts

Custom Object Information

The singular and plural labels are used in tabs, page layouts, and reports. Be careful when changing the name or label as it may affect existing integrations and merge templates.

Label Example: Account

Plural Label Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object via the API.

Object Name Example: Account

Description

Context-Sensitive Help Setting ☒ Open the standard Salesforce.com Help & Training window ☐ Open a window using a Visualforce page

Content Name

Enter Record Name Label and Format

The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the

Setup > Object Manager > Claim

Details

- Fields & Relationships
- Page Layouts
- Lightning Record Pages
- Buttons, Links, and Actions
- Compact Layouts
- Field Sets
- Object Limits
- Record Types
- Related Lookup Filters
- Search Layouts

Record Name: Example: Account Name

Data Type: Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

Display Format: Example: A-{0000} [What is this?](#)

Optional Features

- ☒ Allow Reports
- ☒ Allow Activities
- ☒ Track Field History
- ☐ Allow in Chatter Groups
- ☐ Enable Licensing [i](#)

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more.](#)

- ☒ Allow Sharing
- ☒ Allow Bulk API Access
- ☒ Allow Streaming API Access

Deployment Status [What is this?](#)

- ☐ In Development
- ☒ Deployed

6. Define Record Types on Claim Object:

- Go to setup, Click on Object Manager.
- Search for Claim object and click on Claim__c.
- In the left side panel, click on the record types.
- Click on New.
- Fill out the following details:
 - Select the existing record type as Master.
 - Give record type label as “Accident”
 - Record Type Name will be auto populated.
 - Give a meaningful description.
 - Check the active checkbox.
 - Check the Make Available checkbox for all the profiles.
- Click on Next.
- It will redirect to the Assign page layouts page, Leave everything as is, and click on save.

Setup > Object Manager > Claim

Details

- Fields & Relationships
- Page Layouts
- Lightning Record Pages
- Buttons, Links, and Actions
- Compact Layouts
- Field Sets
- Object Limits
- Record Types**
- Related Lookup Filters
- Search Layouts

Edit Record Type: Accident

Enter a new name for the selected record type and click Save.

Record Type

Record Type Label:

Record Type Name:

Namespace Prefix:

Description:

Active: ☒

[Save](#) [Cancel](#)

//Repeat the same steps for creating the property and life record types as well.

7. Create Fields in the Claim Object:

S.No	Field Name	Data Type	Required
1	Adjuster	Lookup(User)	
2	Approval Status	Picklist(New, Submitted for Approval, Approved, Rejected)	
3	Claim Amount	Currency(16, 2)	
4	Date of Loss	Date/Time	
5	Description	Long Text Area	
6	Policy	Lookup(Policy)	

8. Create an AutoQuoting Screen Flow

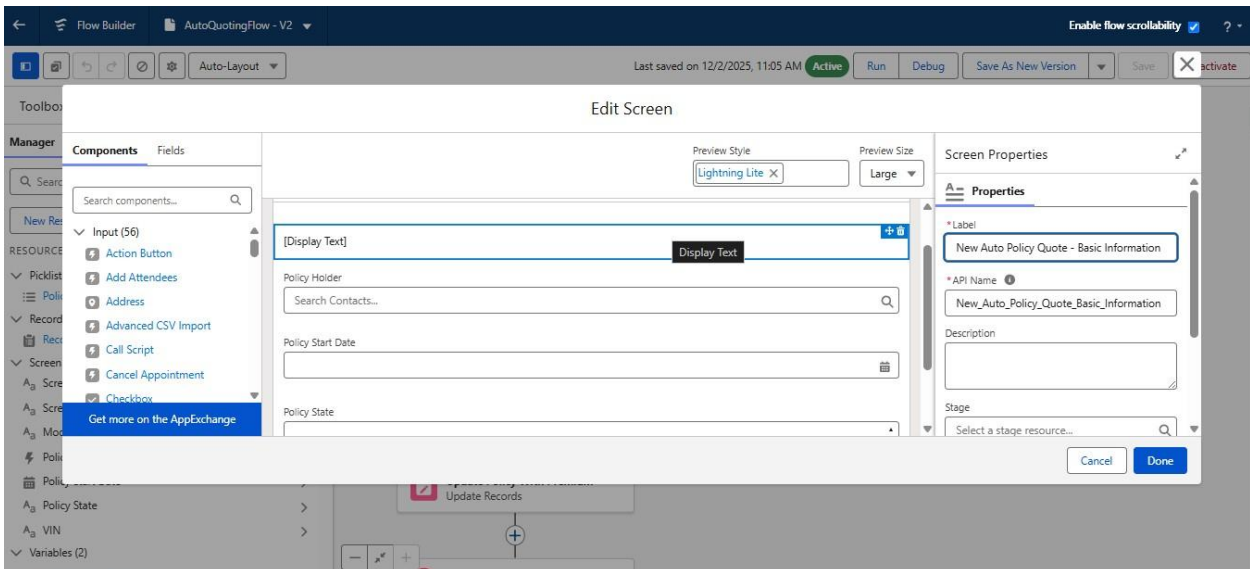
- Go to setup, Search for flows in the quick find box.
- Click on flows and click on new flow.
- Select Screen Flow and Click on Create.
- Add Get Records Element immediately after the start element.
- Give Label as **Get Vehicle RT ID**
- Api Name will be auto populated
- Select Record Type for Object
- For Conditional Requirements, Select All Conditions are met • The Conditions include:
 - sObject Type Name Equals Policy__c
 - And
 - Record Type Name Equals Auto

The image displays two screenshots of the Salesforce Flow Builder interface for a flow named 'AutoQuotingFlow - V2'.

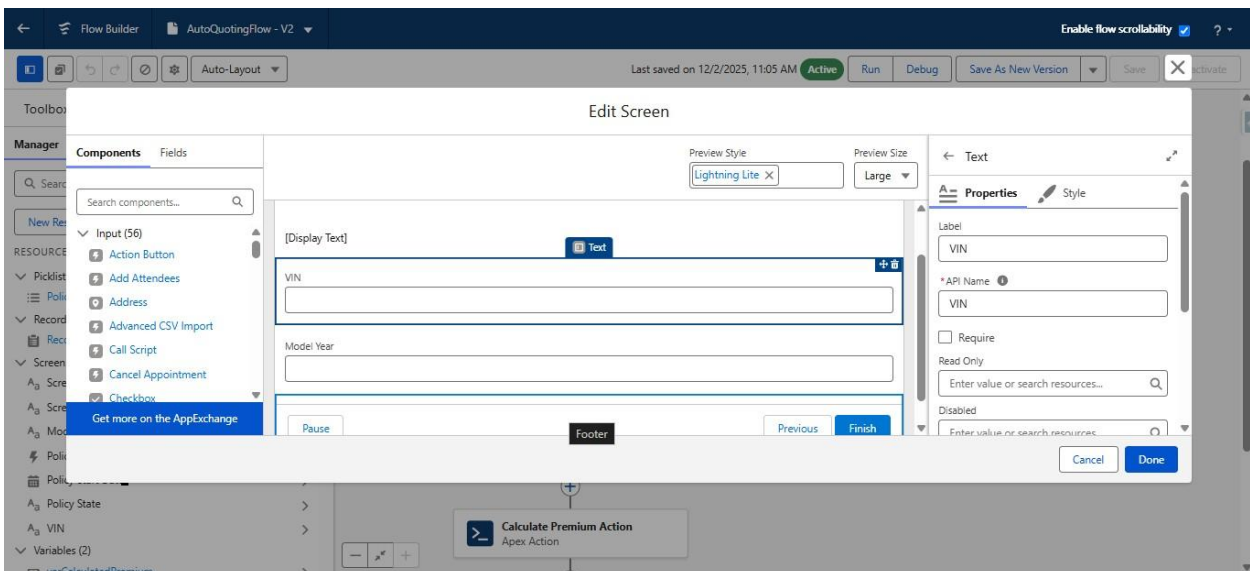
Top Screenshot: The flow canvas shows a sequence of steps: 'Get Vehicle RT ID' (Get Records), 'New Auto Policy Quote - Basic Information' (Screen), 'Vehicle-Specific Details' (Screen), 'Create Draft Policy' (Create Records), and 'Calculate Premium Action' (Apex Action). The right-hand pane shows the configuration for the 'Get Records' step. It includes fields for 'Label' (Get Vehicle RT ID), 'API Name' (Get_Vehicle_RT_ID), and 'Object' (Record Type). The 'Filter Record Type Records' section shows a condition: 'SObject Type Name' equals 'Policy__c' AND 'Record Type Name' equals 'Auto'.

Bottom Screenshot: This screenshot shows the same flow canvas, but the right-hand pane is expanded to show additional configuration options. The 'Sort Record Type Records' section shows 'Sort Order' set to 'Not Sorted'. The 'How Many Records to Store' section has 'Only the first record' selected. The 'How to Store Record Data' section has 'Automatically store all fields' selected.

- Add Screen Element immediately after the Get Records Element • Give the label as New Auto Policy Quote - Basic Information • API Name will be auto populated.
- Drag the display text component into the flow canvas
- Give the API Name as Screen_1_Policy_Basics
- Drag the Lookup Component into the Canvas, Give API Name as Customer Lookup, Field API Name as Customer__c, Label as Policy Holder, ObjectApiName as Policy__c
- Drag Date component into the canvas, Give Label as Policy Start Date, API Name will be auto populated.
- Drag the picklist component into the canvas, Give the label as Policy State, API Name will be auto populated, For Choice, Create a new Resource, Resource Type should be Picklist Choice Set, Give the API Name, Select the Policy for Object, Select Picklist as Datatype, Select Policy State as Field and Click on save
- Click on Done



- Add another screen element after the Auto Policy Quote - Basic Information screen
- Give the label as Vehicle-Specific Details • API Name will be auto populated.
- Drag the display text component into the flow canvas
- Give the API Name as Screen_2_Vehicle_Details
- Drag the text component into the canvas, Give the label as VIN, API name will be auto populated
- Again, Drag the text component into the canvas, Give the label as Model year, API name will be auto populated. • Click on done.



- Add Create Records Element after the screen element.
- Give the label as **Create Draft Policy**, API name will be auto populated
- Store field values manually
- Select the policy object
- Set fields for the policy object as shown below:

- Customer —> From Basic information screen-> Policy Holder -> Record ID
- Model Year——> Vehicle information screen -> Model Year
- Policy Start Date —> From Basic information screen—>Policy Start Date
- Policy State —> From Basic information screen —> Policy State
- RecordType ID —> From Get Vehicle RT Id —> RecordType ID
- VIN —> From Vehicle Information Screen —> VIN
- Check manually assign variables checkbox
- In the place of variable, create a new resource, Whose resource type is variable, API Name would be varPolicyId, Datatype would be Text, Click on Done.

The image displays two screenshots of the Salesforce Flow Builder interface for a flow named "AutoQuotingFlow - V2".

Top Screenshot: The flow is in the "Create Records" step, specifically "Create Draft Policy". The "How to set record field values" is set to "Manually". The "Object" is "Policy". The "Set Field Values for the Policy" section shows the following mappings:

- Field: `A Customer` | Value: `A Basic Information > Policy Holder > Record ID`
- Field: `A Model Year` | Value: `A Vehicle-Specific Details > Model Year`

Bottom Screenshot: The flow is in the "Create Records" step, specifically "Create Draft Policy". The "How to set record field values" is set to "Manually". The "Object" is "Policy". The "Set Field Values for the Policy" section shows the following mappings:

- Field: `A Model Year` | Value: `A Vehicle-Specific Details > Model Year`
- Field: `Policy Start Date` | Value: `A Basic Information > Policy Start Date`
- Field: `Policy State` | Value: `A Basic Information > Policy State`
- Field: `A Record Type ID` | Value: `A Type from Get Vehicle RT ID > Record Type ID`
- Field: `A VIN` | Value: `A Vehicle-Specific Details > VIN`

The "Manually assign variables (advanced)" checkbox is checked. The "Store Policy ID in Variable" section shows the variable `varPolicyId` assigned to the Policy ID field.

- Save the flow with AutoQuotingFlow
- Activate the flow

9. Create Validation Rule on Policy Object:

- Go to object manager, Search for the policy object and click on policy object
- In the left side panel, scroll down and find validation rules and click on it ● Click on New

The screenshot shows the Salesforce Setup interface. The breadcrumb trail is **SETUP > OBJECT MANAGER**. The main heading is **Policy**. On the left sidebar, the **Validation Rules** option is selected. The main content area displays a table titled **Validation Rules** with 1 item. The table has columns: **RULE NAME**, **ERROR LOCATION**, **ERROR MESSAGE**, **ACTIVE**, and **MODIFIED BY**. The single rule listed is **VIN_Must_Be_17_Characters** with error location **VIN** and message **The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.** The **ACTIVE** checkbox is checked. The **MODIFIED BY** is **Insurance Portal, 12/1/2025, 8:50 PM**. A **New** button is in the top right corner of the table.

RULE NAME	ERROR LOCATION	ERROR MESSAGE	ACTIVE	MODIFIED BY
VIN_Must_Be_17_Characters	VIN	The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.	☒	Insurance Portal, 12/1/2025, 8:50 PM

<https://orgfarm-2d73826e88-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/011gK00000308i9/ValidationRules/03dgk000000YIIFCAW/view>

- Give the rule name as VIN_Must_Be_17_Characters
- Check the active checkbox
- Write the following formula in the formula editor
 - `AND( RecordType.DeveloperName = "Auto", LEN( VIN__c )<> 17)`
- For error message, The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.
- Select error location as field and select VIN field
- Click on Save

The screenshot shows the detail page for the **Policy Validation Rule**. The breadcrumb trail is **SETUP > OBJECT MANAGER**. The main heading is **Policy**. On the left sidebar, the **Policy Validation Rule** option is selected. The main content area displays the **Validation Rule Detail** for the rule **VIN_Must_Be_17_Characters**. The **Active** checkbox is checked. The **Error Condition Formula** is `AND( RecordType.DeveloperName = "Auto", LEN( VIN__c )<> 17)`. The **Error Message** is **The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.** The **Error Location** is **VIN**. The **Description** is blank. The **Created By** is **Insurance Portal, 12/1/2025, 8:50 PM** and the **Modified By** is **Insurance Portal, 12/1/2025, 8:50 PM**. There are **Edit** and **Clone** buttons for the rule name, formula, error message, and description.

Validation Rule Detail	
Rule Name	VIN_Must_Be_17_Characters
Error Condition Formula	AND(RecordType.DeveloperName = "Auto", LEN(VIN__c)<> 17)
Error Message	The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.
Error Location	VIN
Description	
Created By	Insurance Portal, 12/1/2025, 8:50 PM
Modified By	Insurance Portal, 12/1/2025, 8:50 PM

Milestone 2: Complex Policy Issuance & Claim Routing Automation

1. Create an Apex Class (PremiumCalculator) with a method that accepts policy data

- Click on setup icon, Right click on developer console, Open the link in new tab
- At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as PremiumCalculator and click on create. • Paste the following code in the apex class:

```
public class PremiumCalculator {
```

```
    public class PremiumInputWrapper {
```

```
        @InvocableVariable(required=true)
```

```
        public Id policyId;
```

```
    }
```

```
    @InvocableMethod(label='Calculate Policy Premium' description='Calculates premium based on policy fields (VIN, Model Year, State).')
```

```
    public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
```

```
        List<Decimal> premiumResults = new List<Decimal>();
```

```
        Set<Id> policyIds = new Set<Id>();
```

```
        for (PremiumInputWrapper input : inputs) {
```

```
if (input.policyId != null) {
```

```
    policyIds.add(input.policyId);
```

```
}
```

```
}
```

```
List<Policy__c> policiesToRate = [
```

```
    SELECT Id, VIN__c, Model_Year__c, Policy_State__c
```

```
    FROM Policy__c
```

```
    WHERE Id IN :policyIds
```

```
];
```

```
for (Policy__c policy : policiesToRate) {
```

```
    Decimal basePremium = 1000.00;
```

```
    if (policy.Policy_State__c == 'CA') {
```

```
        basePremium *= 1.15;
```

```
    } else if (policy.Policy_State__c == 'TX') {
```

```
        basePremium *= 1.05;
```

```
}
```

```
        if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) < 2018) {
```

```
            basePremium += 100;
```

```
        }
```

```
        premiumResults.add(basePremium);
```

```
    }
```

```
    return premiumResults;
```

```
}
```

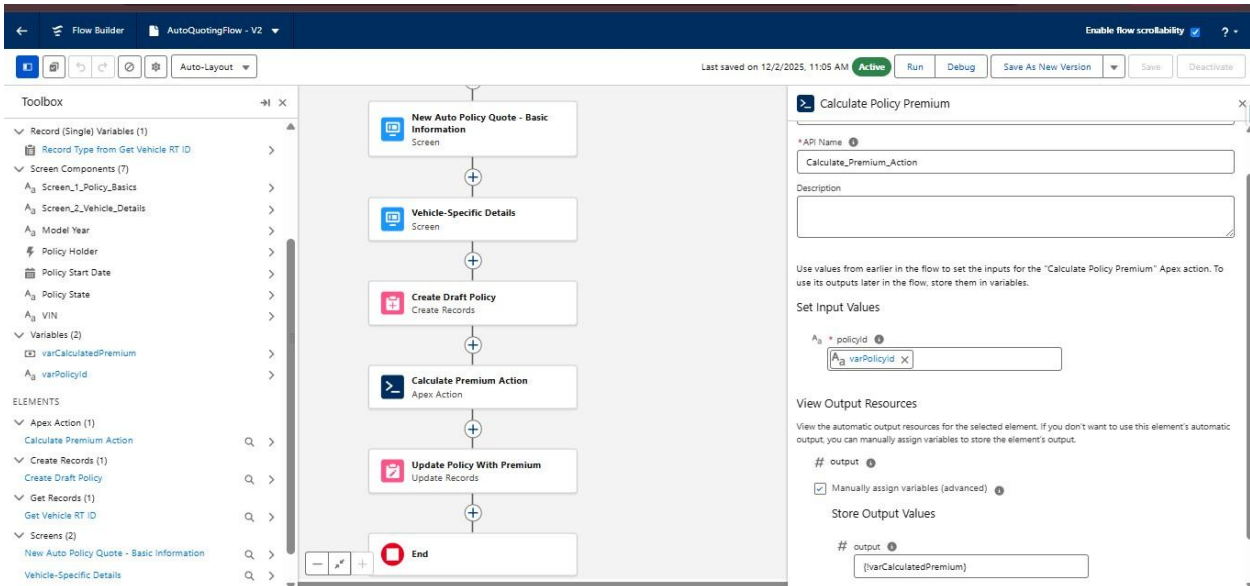
```
}
```

- Click on CTRL + S to save the apex class
- And this apex class need to be invoked in the AutoQuotingFlow

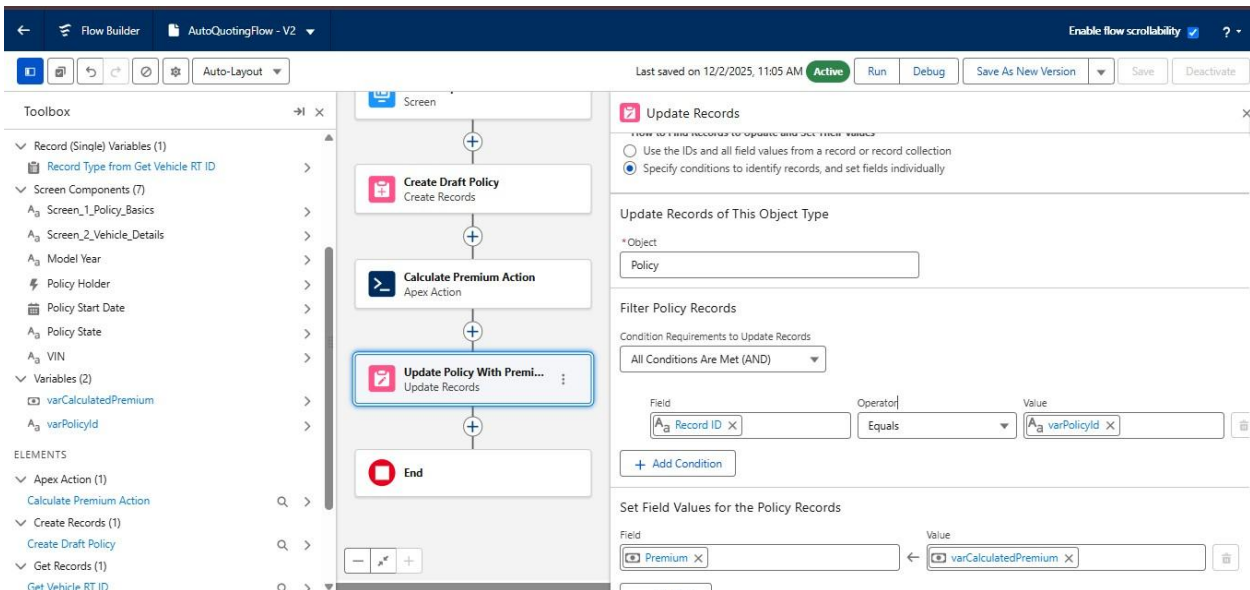
2. Integrate this Apex action into the final step of the *AutoQuotingFlow* to update the draft Policy record

- Go to setup, in the quick find box search for flows and click on flows
- Find for AutoQuotingFlow and open in the flow builder
- Under the Create Records element,
- Add an action element after the create records element.
- Give the label as Calculate Premium Action

- API Name will be auto populated
- Under set Input values, For policy Id, Set the varPolicyId variable created in the previous step
- Under view output resources, check the manually assign variables checkbox and for the output placeholder, create a resource
- Resource type would be variable, API Name would be varCalculatedPremium, Data type would be currency, Decimal places are '2' and click Done.



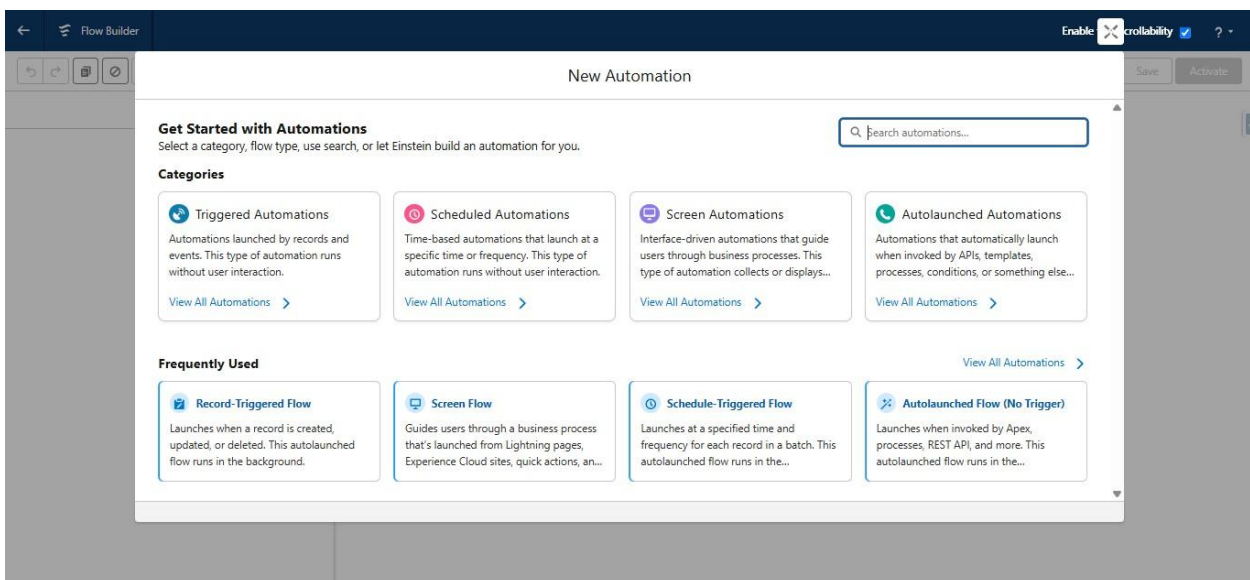
- After the action element, add the update records element, Give the label as Update Policy With Premium
- For how to find records to update and set their values, Select Specify conditions to identify records, and set fields individually
- Select object as policy
- Select all conditions are met for filter criteria
- The conditions include:
 - Record Id Equals varPolicyId variable
 - Under set field values for policy record
 - Assign varCalculatedPremium variable to the Premium field



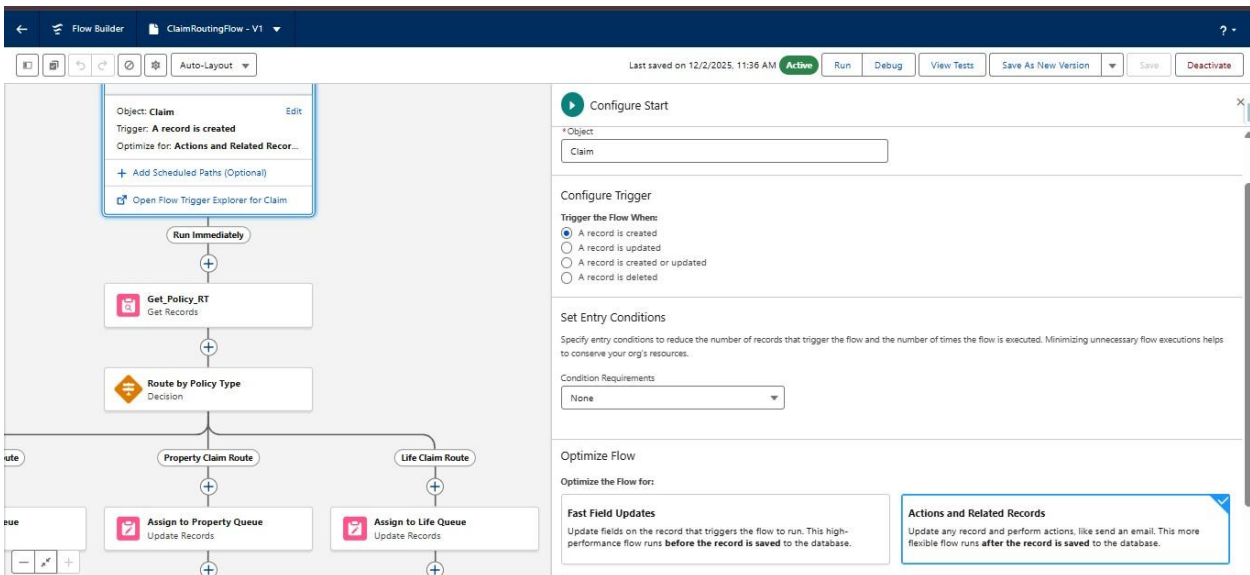
- Save the flow as a new version
- Activate the flow.

3. Create a Record-Triggered Flow (After Save) on the Claim object.

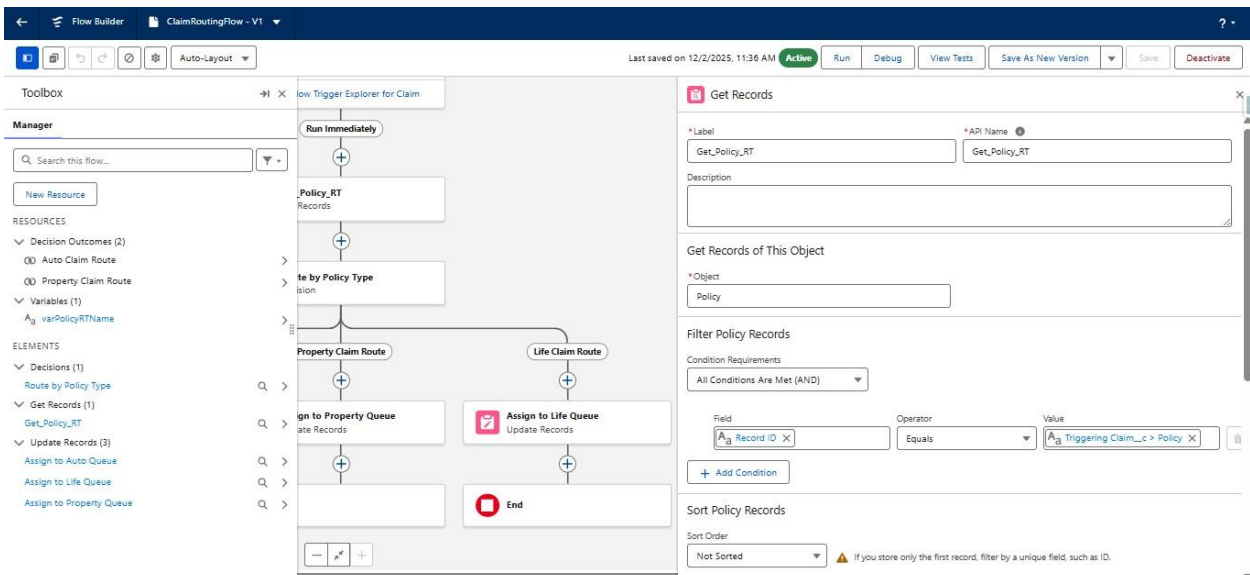
- Go to setup, In the quick find box, Search for flows and click on flows
- Click on New Flow, select Record Triggered Flow and Click on Create

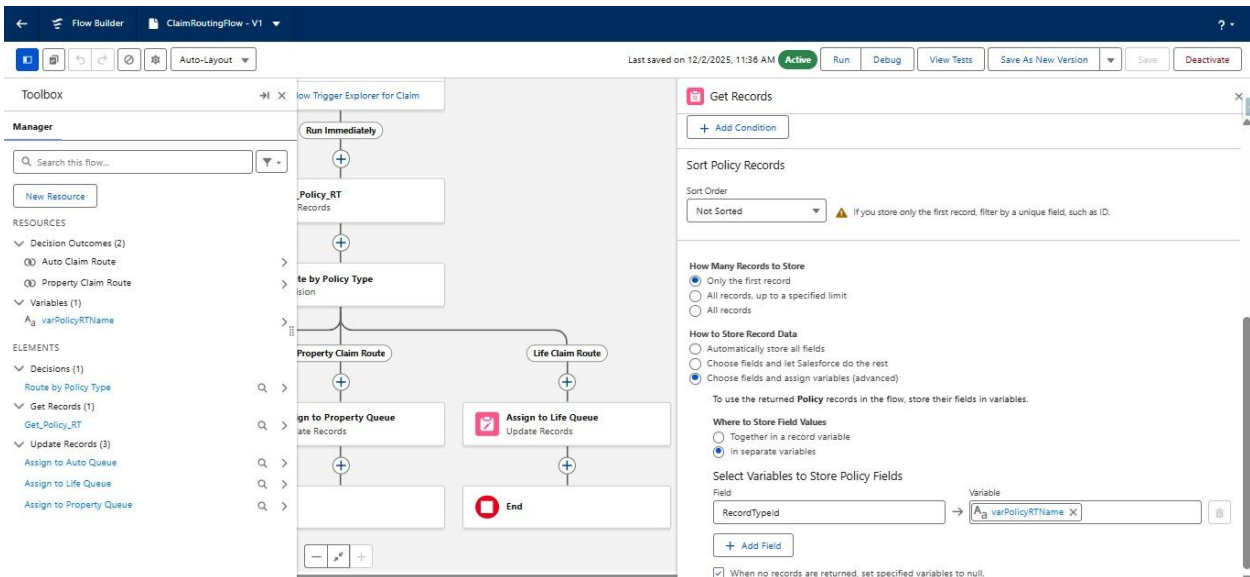


- Select Claim for an object
- Configure trigger when the record is created
- Optimize flow for action and Related Records



- Add Get Records element after the start and give the label as Get_Policy_RT
- Select policy for object
- Specify all conditions are met for conditional requirements • The conditions include:
 - Record Id Equals Triggering Claim → Policy
 - Store only the first record
 - Choose fields and assign variables for storing record data
 - Select in Separate variables for where to store field values section
 - Under select variables to store policy fields
 - For recordTypeID field Create a resource, Resource type would be variable, API Name would be varPolicyRTName, Data type would be Text and click on Done





- Under Get Records Element, Add decision element
- Give the label as Route by Policy Type
- API name will be auto populated
- Select Decision Logic as Manually set conditions ● Specify the Outcomes:
- Outcome1:

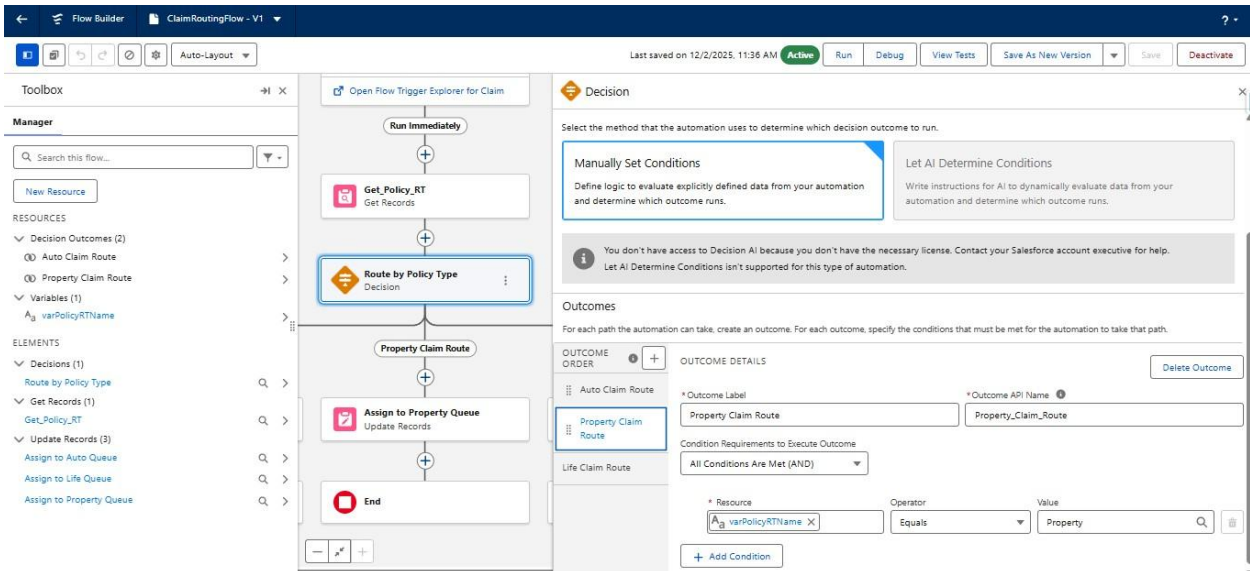
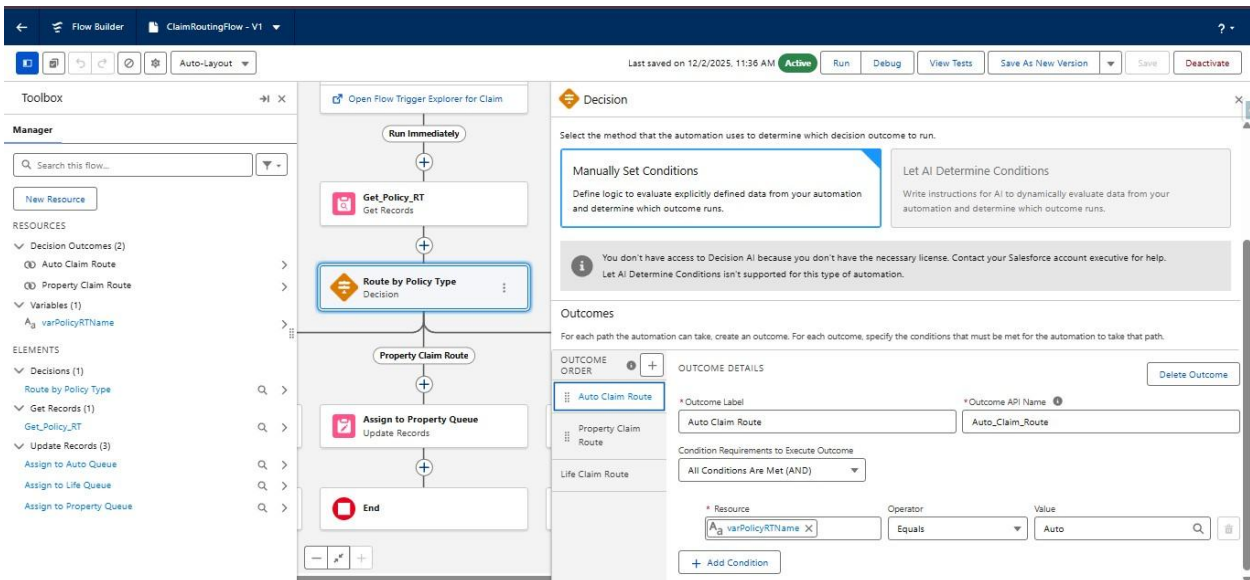
- Label as Auto Claim Route
- API name will be auto populated
- Select All conditions for the conditional Requirements
- The condition includes:
- varPolicyRTName variable equals Auto ●

Outcome2:

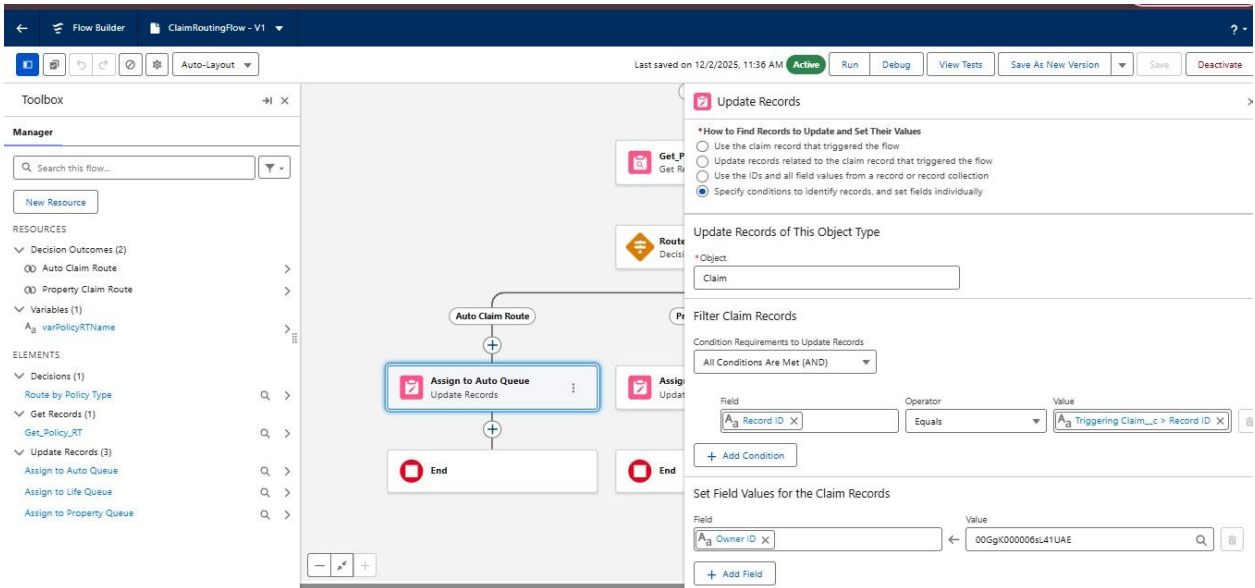
- Label as Property Claim Route
- API name will be auto populated
- Select All conditions for the conditional Requirements ○
- The condition includes:
- varPolicyRTName variable equals Property ●

Default Outcome:

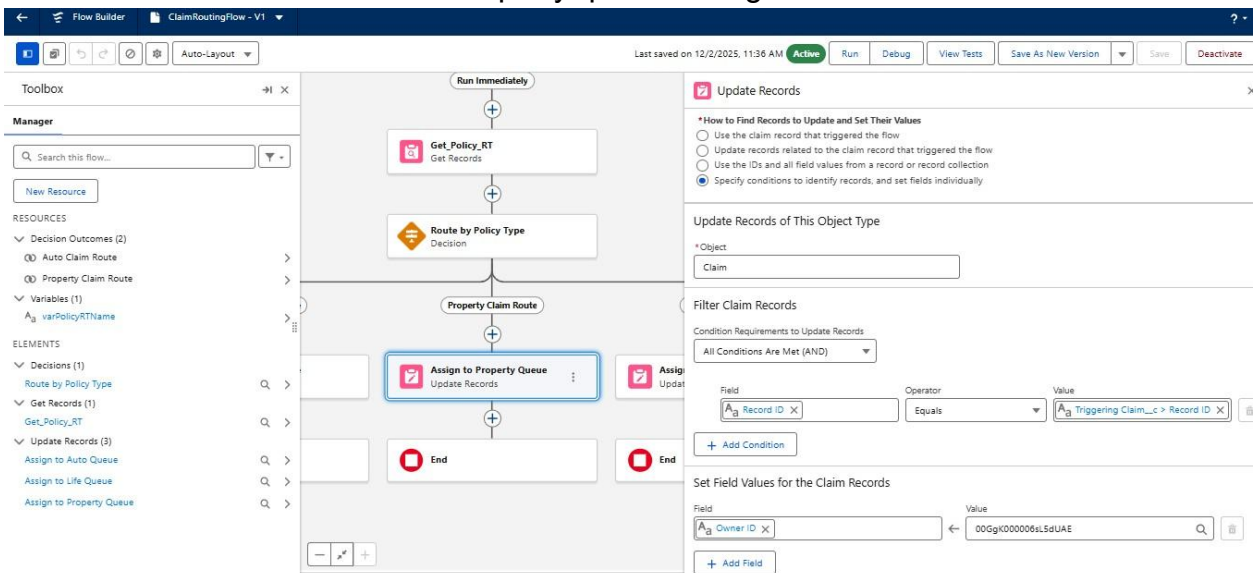
- Label as Life Claim Route



- Under Auto Claim Route Decision path, add the update records element
- Label it as Assign to Auto Queue
- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
 - Record Id equals Triggering Claim → Record ID
 - Under Set fields for the Claim Records
 - Set the hardcoded auto queue 18 digit Id for Owner ID field

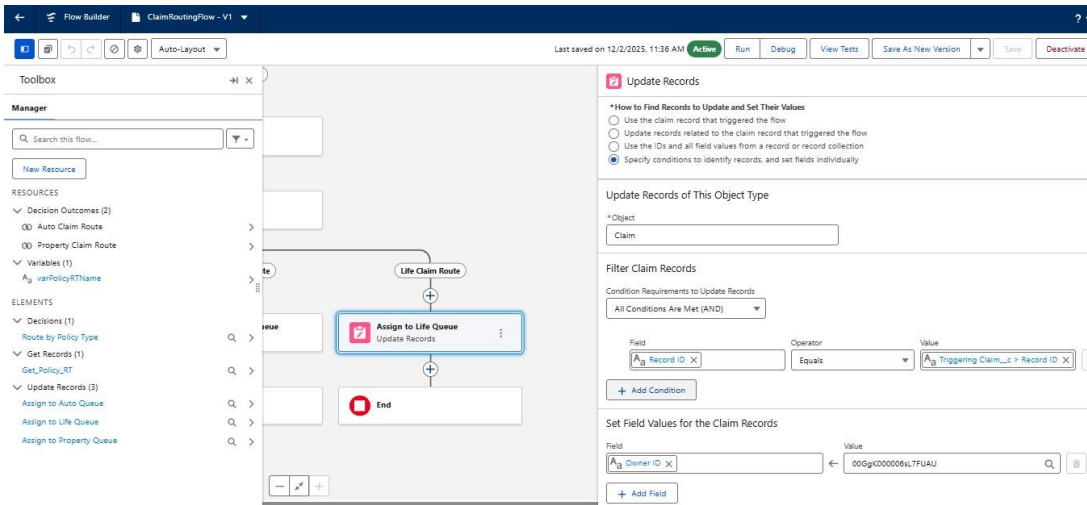


- Under Property Claim Route Decision path, add the update records element
- Label it as Assign to Property Queue
- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
 - Record Id equals Triggering Claim → Record ID
- Under Set fields for the Claim Records
- Set the hardcoded Property queue 18 digit Id for Owner ID field



- Under Life Claim Route Decision path, add the update records element
- Label it as Assign to Life Queue

- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
 - Record Id equals Triggering Claim → Record ID
 - Under Set fields for the Claim Records
 - Set the hardcoded Life queue 18 digit Id for Owner ID field



Milestone 3: Claims Adjuster LWC Dashboard Development

1. Create a robust Apex class (ClaimsAdjusterController) with a method using SOQL to query all Claims owned by the logged-in user, joining related Policy and Customer data. The method must return a custom Wrapper Class listing key metrics.
 - Click on setup icon, Right click on developer console, Open the link in new tab
 - At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as ClaimsAdjusterController and click on create. • Paste the following code in the apex class:

```
public class ClaimsAdjusterController {
```

```
    public class ClaimWrapper {
```

```
        @AuraEnabled public Id claimId;
```

```
        @AuraEnabled public String claimNumber;
```

```
@AuraEnabled public Decimal claimAmount;
```

```
@AuraEnabled public String status;
```

```
@AuraEnabled public String policyType;
```

```
@AuraEnabled public String policyHolderName;
```

```
@AuraEnabled public Integer daysOpen;
```

```
public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
```

```
    this.claimId = c.Id;
```

```
    this.claimNumber = c.Name;
```

```
    this.claimAmount = c.Claim_Amount__c;
```

```
    this.status = c.Approval_Status__c;
```

```
    Policy__c policyRecord = (Policy__c)p;
```

```
    if (policyRecord != null && policyRecord.RecordType != null) {
```

```
        this.policyType = policyRecord.RecordType.Name;
```

```
    } else {
```

```
        this.policyType = 'Unknown';
```

```
    }
```

```
        this.policyHolderName = (customerRec != null) ? customerRec.Name : "";
        this.daysOpen = (c.Date_of_Loss__c != null)
```

```
            ? (Date.today().daysBetween(c.Date_of_Loss__c.date()))
```

```
            : 0;
```

```
    }
```

```
}
```

```
@AuraEnabled(cacheable=true)
```

```
public static List<ClaimWrapper> getAssignedClaims() {
```

```
    List<ClaimWrapper> results = new List<ClaimWrapper>();
```

```
    List<Claim__c> claims = [
```

```
        SELECT
```

```
            Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
```

```
            Policy__r.Name,
```

```
            Policy__r.RecordType.DeveloperName,
```

```
            Policy__r.Customer__r.FirstName,
```

```
Policy__r.Customer__r.LastName
```

```
FROM Claim__c
```

```
WHERE OwnerId = :UserInfo.getUserId()
```

```
ORDER BY Date_of_Loss__c DESC
```

```
];
```

```
for (Claim__c claim : claims) {
```

```
    Policy__c policy = claim.Policy__r;
```

```
    Contact customer = (policy != null && policy.Customer__r != null)
```

```
        ? (Contact)policy.Customer__r
```

```
        : new Contact();
```

```
    results.add(new ClaimWrapper(claim, policy, customer));
```

```
}
```

```
return results;
```

```
}
```

}

- Save the file using CTRL + S
- This apex class will be handled in the LWC component

2. Create a component (claimsDashboardLwc) that uses the Apex method via the @wire service and implements client-side data filtering

- Open the new tab in chrome and search for lightning studio extension
- Open the first link and click on add to chrome, and confirm by clicking add Extension button
- Now open the Lightning studio extension from your salesforce org
- On the Top left corner, you will find Create New
- Click on Create New and Click on Lightning Web Component
- Enter the name of the Component as claimsDashboardLwc
- Check the isExposed checkbox
- Under target configurations, Select Lightning app page, Home page and Record page
- Click on Deploy
- Open the HTML file and paste the following code:

```
<template>

    <lightning-card title="My Assigned Claims Dashboard"
    icon-name="standard:claim">
```

```
<div class="slds-m-around_medium">

    <lightning-combobox

        name="policyFilter"

        label="Filter by Policy Type"

        value="All"
```

```

        options={policyTypeOptions}

        onchange={handleFilterChange}

        class="slds-m-bottom_medium"

></lightning-combobox>

<template if:true={visibleClaims}>

    <div class="slds-grid slds-wrap slds-grid_pull-padded">

        <template for:each={visibleClaims} for:item="claim">

            <div key={claim.claimId} class="slds-col
slds-size_1-of-1 slds-medium-size_1-of-2 slds-large-size_1-of-3
slds-p-horizontal_small">

                <c-claim-tile-lwc
claim-data={claim}></c-claim-tile-lwc>

            </div>

        </template>

    </div>

    <p class="slds-m-top_medium slds-text-heading_small">Total
Claims Assigned: {visibleClaims.length}</p>

</template>

```

```

        <template if:false={visibleClaims}>

            <p>No claims currently assigned to you.</p>

        </template>

    </div>

</lightning-card>

</template>

```

- Open the JS file and paste the following code:

```

import { LightningElement, wire, track } from 'lwc';
// Import the Apex method

import getAssignedClaims from
'@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';

```

```

export default class ClaimsDashboardLwc extends LightningElement {

    // List to hold all data retrieved from Apex

    @track allClaims = [];

    // List to display after filtering

```

```
@track visibleClaims = [];
```

```
error;
```

```
// Wire service to call the Apex method
```

```
@wire(getAssignedClaims)
```

```
wiredClaims({ error, data }) {
```

```
    if (data) {
```

```
        this.allClaims = data;
```

```
        this.visibleClaims = data; // Initially display all claims
```

```
        this.error = undefined;
```

```
    } else if (error) {
```

```
        this.error = error;
```

```
        this.allClaims = undefined;
```

```
        this.visibleClaims = undefined;
```

```
        console.error('Error retrieving claims:',  
JSON.stringify(error));
```

```
    }
```

```
}

// Client-side filtering logic (Required by M3)

handleFilterChange(event) {

    const selectedPolicyType = event.target.value;

    if (selectedPolicyType === 'All') {

        this.visibleClaims = this.allClaims;

    } else {

        // Filter claims where policyType matches the selected value

        this.visibleClaims = this.allClaims.filter(claim =>

            claim.policyType === selectedPolicyType

        );

    }

}

get policyTypeOptions() {
```

```

// Options for the filter dropdown

return [

    { label: 'All Policy Types', value: 'All' },

    { label: 'Auto', value: 'Auto' },

    { label: 'Property', value: 'Property' },

    { label: 'Life', value: 'Life' }

];

}

}

```

- Open the meta-xml file and paste the following code:

```

<?xml version="1.0"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>

    <isExposed>true</isExposed>

    <targets>

        <target>lightning__AppPage</target>

        <target>lightning__HomePage</target>
    
```

```
</targets>

</LightningComponentBundle>
```

3. Create a reusable component (claimTileLwc) to display essential information for each claim

- Open the Lightning studio extension from your salesforce org
- On the Top left corner, you will find Create New
- Click on Create New and Click on Lightning Web Component
- Enter the name of the Component as **claimTileLwc**
- Check the isExposed checkbox
- Under target configurations, Select Lightning app page, Home page and Record page
- Click on Deploy
- Open the HTML file and paste the following code:

```
<template>

    <lightning-layout-item size="12" padding="around-small">

        <div class="tile slds-box slds-p-around_medium
slds-theme_default">
            <header class="slds-
media slds-media_center

slds-m-bottom_small">

                <div class="slds-media__figure">

                    <lightning-icon icon-name={policyIcon}
alternative-text={claimData.policyType} size="small"></lightning-icon>

                </div>
```

```
<div class="slds-media__body slds-m-left_small">

    <h2 class="slds-text-heading_medium
slds-m-bottom_none">

        Claim: {claimData.claimNumber}

    </h2>

    <p class="slds-text-body_small">Policy Type:
<strong>{claimData.policyType}</strong></p>

</div>

</header>
```

```
<div class="slds-border_top slds-p-top_small">

    <p>Policy Holder:
<strong>{claimData.policyHolderName}</strong></p>

    <lightning-formatted-number

        value={claimData.claimAmount}

        format-style="currency"

        currency-code="USD"
```

```

        label="Claim Amount">

        </lightning-formatted-number>

        <p>Status: {claimData.status}</p>

        <p class="slds-text-body_regular">

            Days Open: <strong
class="slds-text-color_error">{claimData.daysOpen}</strong>

        </p>

    </div>

</div>

</lightning-layout-item>

</template>

```

- Open the JS File and paste the following code

```

import { LightningElement, api } from 'lwc';

export default class ClaimTileLwc extends LightningElement {

```

```

// @api decorator exposes this property to the parent component

@api claimData;

// Helper property to determine the correct icon based on Policy Type

get policyIcon() {

    if (this.claimData && this.claimData.policyType) {

        const type = this.claimData.policyType.toLowerCase();

        if (type === 'auto') return 'utility:truck';

        if (type === 'property') return 'utility:home';

        if (type === 'life') return 'utility:connected_apps';

    }

    return 'utility:help';

}

}

```

- Open the meta-xml file and paste the following code

```

<?xml version="1.0"?>

```

```
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>

    <isExposed>true</isExposed>

</LightningComponentBundle>
```

Milestone 4: Advanced Claim Processing, Security & Code Setup

1. Modify the Claim object to include an Approval Status field.

- Go to set up, Click on object manager, search for claim object and click on claim object.
- In the left side panel, click on fields and relationships.
- Click on new
- Select Datatype as picklist
- Label it as Approval Status
- Select enter values radio button
- Enter New, Submitted for Approval, Approved and Rejected line by line • Field Name will be auto populated • Click on Next, Next and Save.

2. Configure the Approval Process to route first to the Senior Adjuster, then to the Department Manager, using two sequential steps.

- Go to setup, In the quick find box, Search for Approval process
- Select claim object for Manage approval process for
- Click on create new Approval Process and select use standard setup wizard
- Give the process name as High Value Claim Approval
- Unique Name will be auto populated
- Click on Next

The screenshot shows the Salesforce Setup interface. The left sidebar contains a search bar with 'approval' and a list of categories: Data, Feature Settings, and Process Automation. Under 'Process Automation', 'Approval Processes' is selected. The main content area is titled 'Approval Processes' and shows the 'High Value Claim Approval' process. It is at 'Step 1 of 6: Enter Name and Description'. The form fields are: Process Name (High Value Claim Approval), Unique Name (High_Value_Claim_Approval), and Description (empty). There are 'Save', 'Next', and 'Cancel' buttons at the bottom right.

- Under specify entry criteria, Select criteria are met for Use this approval process if
- Criteria includes:
- Claim amount greater than 50000
- Click on Next

The screenshot shows the same Salesforce Setup interface, but at 'Step 2 of 6: Specify Entry Criteria'. The form is titled 'Specify Entry Criteria' and includes a dropdown menu set to 'criteria are met'. Below this is a table for defining criteria:

Field	Operator	Value	AND
Claim: Claim Amount	greater than	50000	AND
--None--	--None--		AND
--None--	--None--		AND
--None--	--None--	Value 3	AND
--None--	--None--		

There is an 'Add Filter Logic...' link below the table. At the bottom right, there are 'Previous', 'Save', 'Next', and 'Cancel' buttons.

- Under Record Editability properties, Select Administrators ONLY can edit the records during the approval process
- Click on Next

Setup Home Object Manager

Search Setup

approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Approval Process Edit

High Value Claim Approval

Step 3. Specify Approver Field and Record Editability Properties

Step 3 of 6

When you define approval steps, you can assign approval requests to different users. One of your options is to use a user field to automatically route these requests. If you want to use this option for any of your approval steps, select a field from the picklist below. Also, when a record is in the approval process, it will always be locked-- only an administrator will be able to edit it. However, you may choose to also allow the currently assigned approver to edit the record.

Select Field Used for Automated Approval Routing

Next Automated Approver Determined By: --None--

Use Approver Field of Claim Owner: ☐

Record Editability Properties

☒ Administrators ONLY can edit records during the approval process.

☐ Administrators OR the currently assigned approver can edit records during the approval process.

Previous Save Next Cancel

- Click on next again
- Under Approval Page Layout, Select Claim Number, Owner, Claim Amount, Adjuster, Policy fields to the selected fields from available fields
- Under security settings, Select Allow Approvers to access the page only from within the salesforce application.
- Click on Next

Setup Home Object Manager

Search Setup

approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Approval Process Edit

High Value Claim Approval

Step 4. Approval Page Layout and Security Settings

Step 4 of 6

Approval Page Fields

Available Fields: Approval Status, Created By, Date of Loss, Description, Last Modified By, Record Type

Selected Fields: Claim Number, Owner, Claim Amount, Adjuster, Policy

Up Down

Security Settings

☒ Allow approvers to access the approval page only from within the Salesforce application. (Recommended)

☐ Allow approvers to access the approval page from within the Salesforce application, or externally from a wireless-enabled mobile device.

Click here to view an example

Previous Save Next Cancel

- Under Initial Submitters, Search for owner under submitter type, and select the claim owner as allowed submitters from available submitters
- Click on Save

Setup Home Object Manager

Search Setup

approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Submitter Type Search: Owner for Find

Available Submitters

-None-

Allowed Submitters

Claim Owner

Add

Remove

Submission Settings

☐ Allow submitters to recall approval requests

Previous Save Cancel

2.1 Create Approval Steps:

Step1:

- Under initial submission actions, You can find Approval Steps
- Under approval steps, Click on New
- Give the Name as Step1 Senior Adjuster
- Unique name will be auto populated
- Click on Next

Setup Home Object Manager

approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Approval Step Edit Step1 Senior Adjuster

Step 1. Enter Name and Description Step 1 of 3

Save Next Cancel

Enter a name, description, and step number for your new approval step.

Enter Name and Description

Approval Process Name High Value Claim Approval

Name Step1 Senior Adjuster

Unique Name Step1_Senior_Adjuster

Description

Save Next Cancel

- Under specify step criteria, Select all records should enter the step
- Click on Next

The screenshot shows the Salesforce Setup interface. The left sidebar contains a search bar with 'approval' and a navigation menu with categories: Data (Mass Transfer Approval Requests), Feature Settings (Approval Settings), and Process Automation (Approval Processes). The main content area is titled 'Approval Processes' and shows 'Step 1: Senior Adjuster'. Below this is 'Step 2: Specify Step Criteria', which is 'Step 2 of 3'. The instructions state: 'Specify whether a record must meet certain criteria before entering this approval step. If these criteria are not met, the approval process can skip to the next step, if one exists. [Learn more](#)'. The 'Specify Step Criteria' section has two options: 'All records should enter this step.' (selected) and 'Enter this step if the following criteria are met', followed by a dropdown menu showing 'criteria are met' and 'reject record'. Navigation buttons 'Previous', 'Save', 'Next', and 'Cancel' are at the bottom right.

- Under select Approver, Select Automatically assign to approver
- Under that select the Senior Adjuster User
- Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response
- Click on Save

The screenshot shows the Salesforce Setup interface for 'Step 3: Select Assigned Approver', which is 'Step 3 of 3'. The instructions state: 'Specify the user who should approve records that enter this step. Optionally, choose whether the approver's delegate is also allowed to approve these requests.' The 'Select Approver' section has three options: 'Let the submitter choose the approver manually.', 'Automatically assign to queue.', and 'Automatically assign to approver(s).'. The third option is selected. Below it, a table shows a single row with 'User' and 'Senior Adjuster'. There are 'Add Row' and 'Remove Row' links. Below the table, the 'When multiple approvers are selected:' section has two options: 'Approve or reject based on the FIRST response.' (selected) and 'Require UNANIMOUS approval from all selected approvers.'. There is also a checkbox for 'The approver's delegate may also approve this request.' Navigation buttons 'Previous', 'Save', and 'Cancel' are at the bottom right.

- Click on New again to create one more step
- Give the name as Step2 Manager Review
- Unique name will be auto populated
- Click on Next

Setup Home Object Manager

Q approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Save Next Cancel

Enter a name, description, and step number for your new approval step.

Previous Approval Step Information	
Step Number:	1
Name:	Step1 Senior Adjuster
Criteria:	
Assign To:	User : Senior Adjuster

Enter Name and Description ! Required Information

Approval Process Name: High Value Claim Approval

Name:

Unique Name:

Description:

Save Next Cancel

- Under specify step criteria, Select all records should enter the step
- Click on Next

Setup Home Object Manager

Q approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Step 2. Specify Step Criteria Step 2 of 3

Previous Save Next Cancel

Specify whether a record must meet certain criteria before entering this approval step. If these criteria are not met, the approval process can skip to the next step, if one exists. [Learn more](#)

Previous Approval Step Information	
Step Number:	1
Name:	Step1 Senior Adjuster
Criteria:	
Assign To:	User : Senior Adjuster

Specify Step Criteria

☒ All records should enter this step.

☐ Enter this step if the following :

Previous Save Next Cancel

- Under select Approver, Select Automatically assign to approver
- Under that select the Department Manager User
- Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response
- Under Reject behaviour, Select Perform all rejection actions for this step AND all final rejection actions
- Click on Save

Setup Home Object Manager

Q approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Select Approver

☐ Let the submitter choose the approver manually.
☐ Automatically assign to queue.
☒ Automatically assign to approver(s).

User

Add Row Remove Row

When multiple approvers are selected:
☒ Approve or reject based on the FIRST response.
☐ Require UNANIMOUS approval from all selected approvers.

☐ The approver's delegate may also approve this request.

Reject Behavior

What should happen if the approver rejects this request?

☒ Perform all rejection actions for this step AND all final rejection actions. (Final Rejection)
☐ Perform ONLY the rejection actions for this step and send the approval request back to the most recent approver. (Go Back 1 Step)

Previous Save Cancel

- Under Initial Submission Actions, Click on Add New and select Field Update
- Give the name as Update Status Submitted
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select submitted for Approval
- Click on Save

Setup Home Object Manager

Q approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Field Updates

Identification

Name

Unique Name

Description

Object Claim

Field to Update Claim: Approval Status

Field Data Type Picklist

Re-evaluate Workflow Rules after Field Change ☐

Specify New Field Value

Picklist Options

☐ The value above the current one
☐ The value below the current one
☒ A specific value

Save Save & New Cancel

- Under Final Approval Actions, Click on Add New and select Field Update
- Give the name as Update Status Approved
- Unique name will be auto populated

- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select Approved
- Click on Save

The screenshot shows the Salesforce Setup interface. The left sidebar contains a search bar with 'approval' and a navigation menu with categories: Data, Feature Settings, and Process Automation. Under Feature Settings, 'Approval Settings' is selected. The main content area is titled 'Field Updates' and shows the configuration for a field update named 'Update Status Approved'. The 'Name' field is 'Update Status Approved', and the 'Unique Name' is 'Update_Status_Approved'. The 'Object' is 'Claim', and the 'Field to Update' is 'Claim: Approval Status'. The 'Field Data Type' is 'Picklist'. The 'Specify New Field Value' section shows 'Picklist Options' with three radio buttons: 'The value above the current one', 'The value below the current one', and 'A specific value' (which is selected). The 'A specific value' dropdown is set to 'Approved'. At the bottom, there are buttons for 'Save', 'Save & New', and 'Cancel'.

- Under Final Rejection Actions, Click on Add New and select Field Update
- Give the name as Update Status Rejected
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select Rejected
- Click on Save

The screenshot shows the Salesforce Setup interface. The left sidebar contains a search bar with 'approval' and a navigation menu with categories: Data, Feature Settings, and Process Automation. Under Feature Settings, 'Approval Settings' is selected. The main content area is titled 'Field Updates' and shows the configuration for a field update named 'Update Status Rejected'. The 'Name' field is 'Update Status Rejected', and the 'Unique Name' is 'Update_Status_Rejected'. The 'Object' is 'Claim', and the 'Field to Update' is 'Claim: Approval Status'. The 'Field Data Type' is 'Picklist'. The 'Specify New Field Value' section shows 'Picklist Options' with three radio buttons: 'The value above the current one', 'The value below the current one', and 'A specific value' (which is selected). The 'A specific value' dropdown is set to 'Rejected'. At the bottom, there are buttons for 'Save', 'Save & New', and 'Cancel'.

- Activate the Approval Process.

3. Create a Record-Triggered Flow (After Save) that checks if the Claim Amount is \$>\$50,000\$ and submits the record to an Approval Process.

- Go to setup, In the quick find box, search for flows and click on flows.
- Click on New Flow
- Select Record Triggered Flow
- Select object as Claim
- Configure Trigger When a Record is created or updated • Select All conditions are true for conditional requirements • The conditions include:
 - Claim Amount Greater than 50000
 - Approval Status Equals New
- Select Every time a record is updated and meets the conditional requirements for When to run the flow for Updated Records
- Optimize the flow for Action and Related Records

The screenshot displays the Salesforce Flow Builder interface for a flow named "Submission Automation Flow - V1". The flow is configured as a "Record-Triggered Flow" with the following settings:

- Object:** Claim
- Trigger:** A record is created or updated
- Conditions:** 2
- Optimize for:** Actions and Related Records

The flow diagram shows a "Record-Triggered Flow" start node, followed by a "Run Immediately" connector, and then a "Submit Claim to Approval" action node. The flow ends with an "End" node.

The right-hand pane shows the "Configure Start" section with the following options:

- Select Object:** Claim
- Configure Trigger:**
 - Trigger the Flow When:
 - ☐ A record is created
 - ☐ A record is updated
 - ☒ A record is created or updated
 - ☐ A record is deleted
- Set Entry Conditions:** Specify entry conditions to reduce the number of records that trigger the flow and the number of times the flow is executed. Minimizing unnecessary flow executions helps to conserve your org's resources.

A tip at the bottom right states: "If you create a flow that's triggered when a record is updated, we recommend first defining entry conditions. Then select the **Only when a record is updated to meet the condition requirements** option for When to Run the Flow for Updated Records."

Record-Triggered Flow Start

Object: **Claim** Edit

Trigger: **A record is created or updated**

Conditions: **2**

Optimize for: **Actions and Related Records**

[+ Add Scheduled Paths \(Optional\)](#)

[Open Flow Trigger Explorer for Claim](#)

Configure Start

Field: **Claim Amount** Operator: **Greater Than** Value: **50000**

Field: **Approval Status** Operator: **Equals** Value: **A New**

AND

[+ Add Condition](#)

When to Run the Flow for Updated Records

☒ Every time a record is updated and meets the condition requirements

☐ Only when a record is updated to meet the condition requirements

Optimize Flow

Optimize the Flow for:

Fast Field Updates
Update fields on the record that triggers the flow to run. This high-performance flow runs **before the record is saved** to the database.

Actions and Related Records
Update any record and perform actions, like send an email. This more flexible flow runs **after the record is saved** to the database.

- Add an action element after the Start Element
- Label it as Submit Claim to Approval
- Under Set Input Values, For Record Id, Select Triggering Claim → Record ID
- For select Approval Process Name or Id, Enter the Approval Process Name(High Value Claim Approval)

Submit for Approval

Label: **Submit Claim to Approval**

*API Name: **Submit_Claim_to_Approval**

Description:

Use values from earlier in the flow to set the inputs for the "Submit for Approval" core action. To use its outputs later in the flow, store them in variables.

Set Input Values

*** Record ID**

A Triggers Claim_c > Record ID

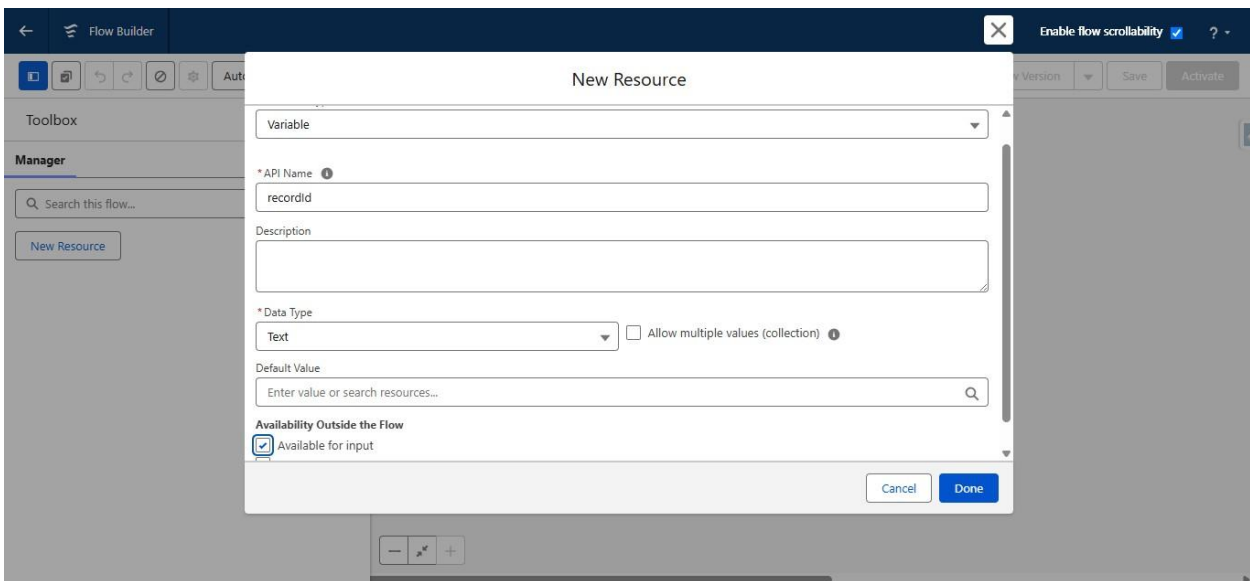
Approval Process Name Or ID

High Value Claim Approval

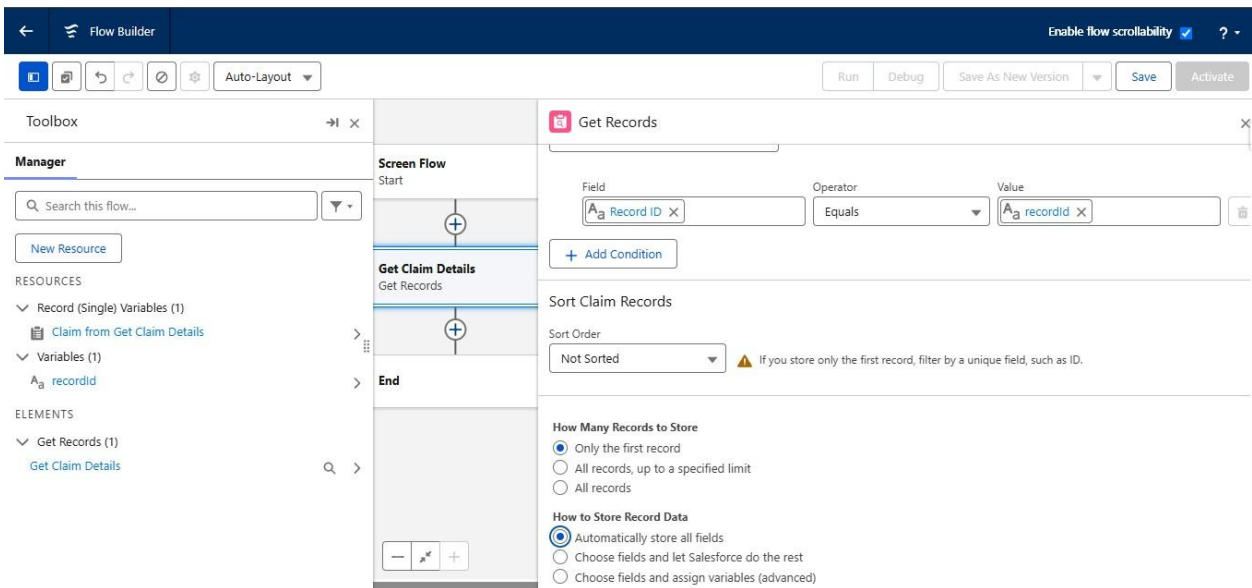
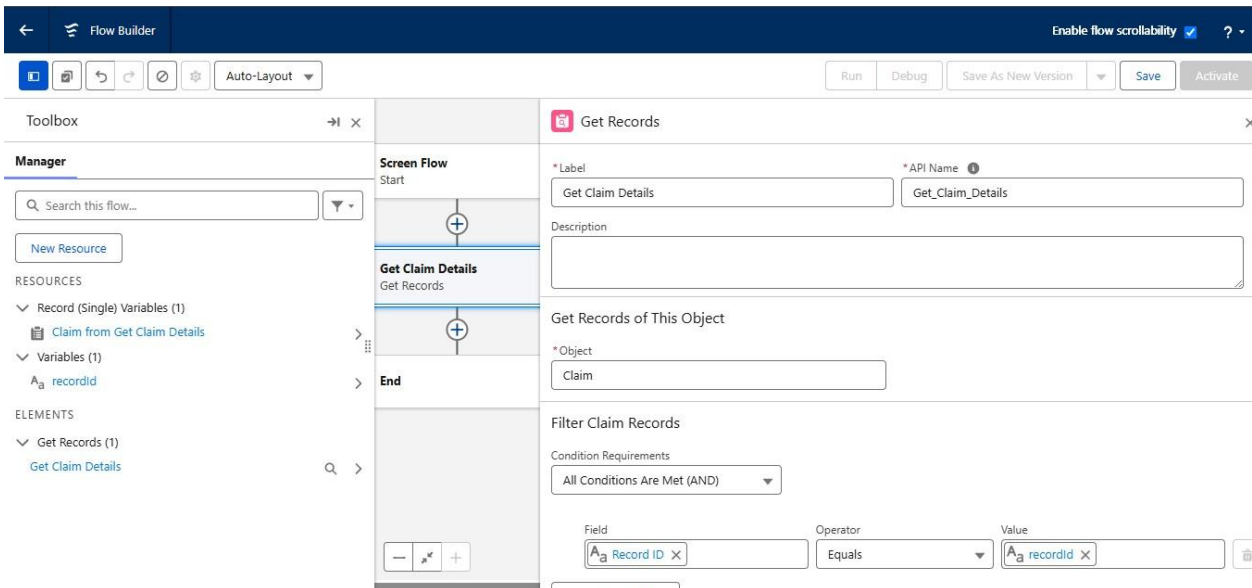
- Save the Flow with Submission Automation Flow and Activate the flow

4. Implement a Screen Flow for the approvers to quickly review details and approve/reject directly from the Claim record.

- Go to setup, Search for flows in the quick find box and click on flows.
- Click on New Flow, Choose Screen Flow
- Create a New Resource by clicking on New Resource button in the toolbox manager
- Resource Type would be Variable
- API Name would be recordId
- Data type would be Text
- Under Availability outside the flow, Check Available for Input • Click on Done

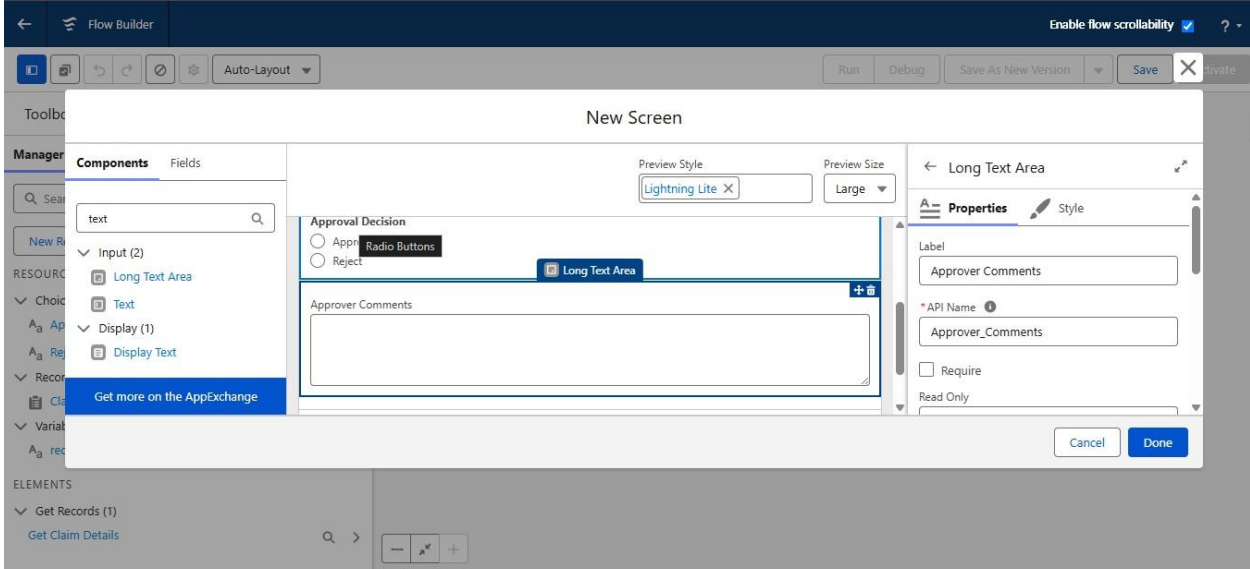
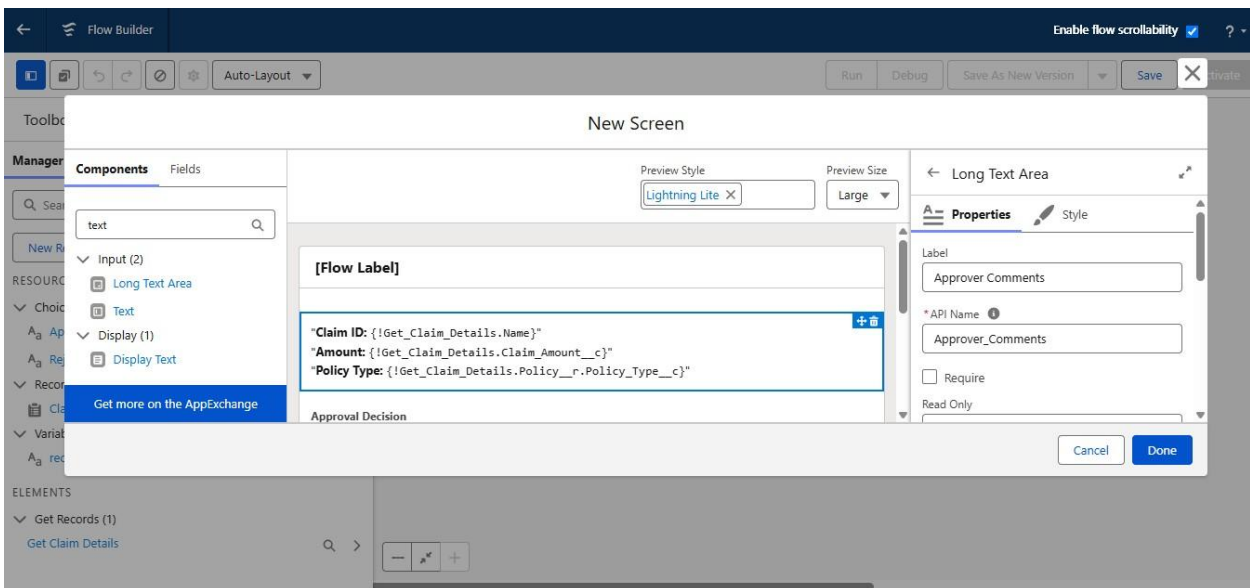


- Add the Get Records Element below the screen
- Label it as Get Claim Details
- Select object as Claim
- Select All conditions are required for conditional Requirements •
The Condition include:
- RecordID equals variable recordId
- Store only the first record and
- Automatically store all fields

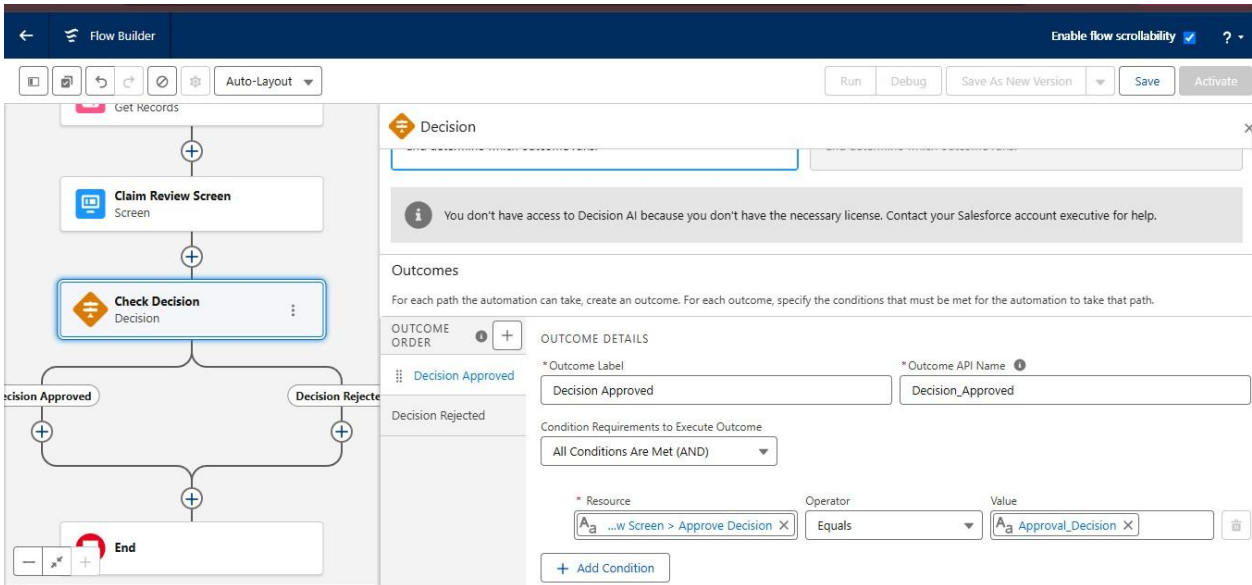


- Add a screen Element under the get Record element
- Label it as Claim Review Screen
- Add Display Text component to the flow canvas, and paste the following:
 - "Claim ID: {!Get_Claim_Details.Name}"
 - "Amount: {!Get_Claim_Details.Claim_Amount__c}"
 - "Policy Type:
 - {!Get_Claim_Details.Policy__r.RecordType.Name}"
- Add the Radio buttons component, Label it as Approval Decision
- API Name would be auto populated
- For Choice, Create a resource, Resource type would be choice
- Give the choice label as Approve
- Select Datatype as Text

- Enter the choice value as Approve
- Enter the input label as Approve
- Click on Done
- Add one more Choice, Create a resource, Resource type would be choice
- Give the choice label as Reject
- Select Datatype as Text
- Enter the choice value as Reject
- Enter the input label as Reject
- Click on Done
- Add Long Text area into the flow canvas and Label it as Approver Comments
- API Name would be Auto Populated
- Click on Done



- Add Decision Element under the screen
- Label it as Check Decision • Select Manually Select Conditions • Set Outcome1:
 - Label it as Decision Approved
 - Select all conditions are met for conditional requirements ◦ The condition includes:
 - Approve Decision From Claim Review Screen → Equals Approval Decision • Set default Outcome:
 - Label it as Decision Rejected



- Add Action Element Under Decision Approved Path
- Search for the submit for approval action and choose it
- Label it as Approve Claim
- Under Set Input Values
- Select variable recordID for Record ID
- Include the submission comments and Enter Approver Comments from Claim Review Screen

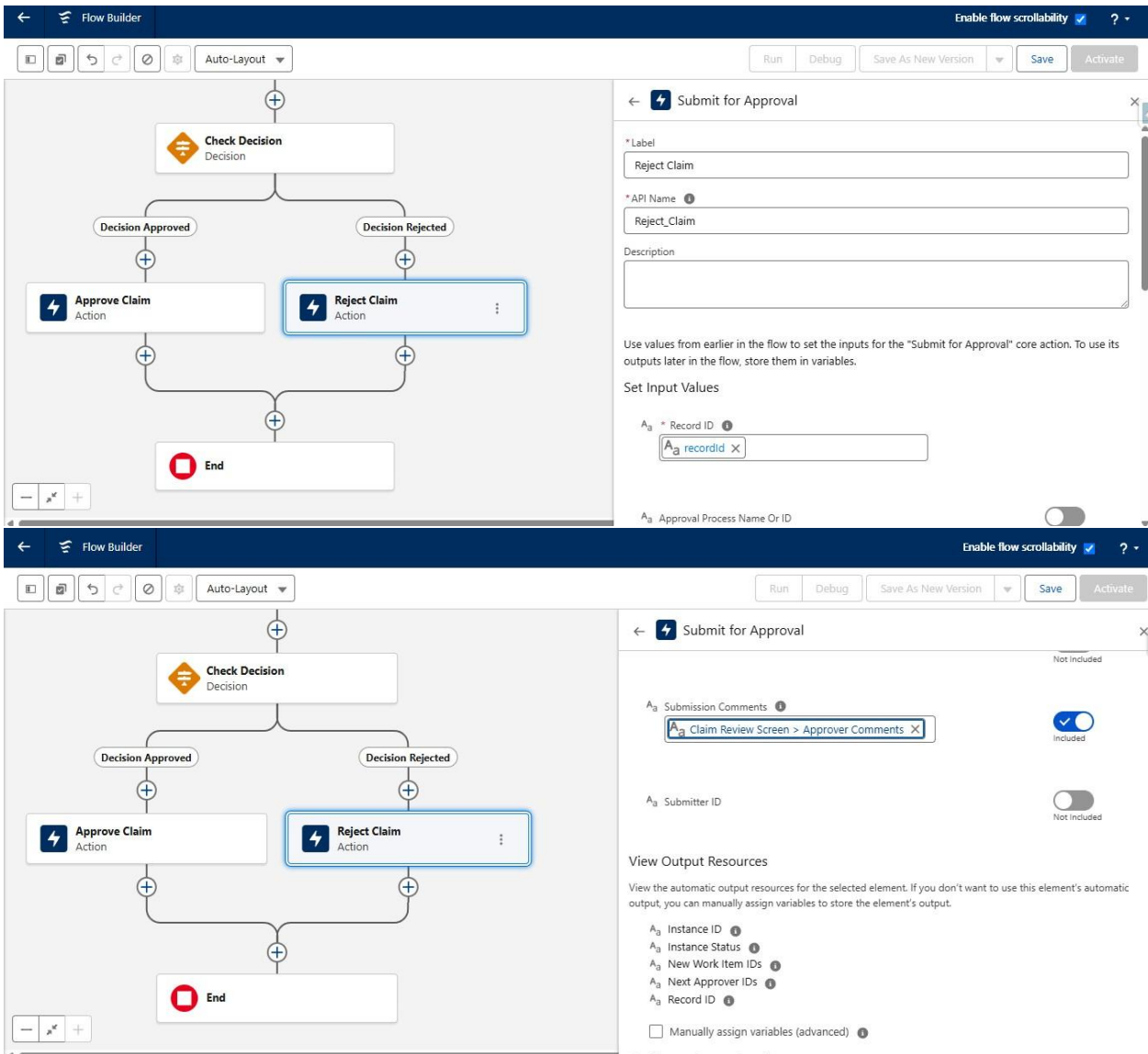
The image displays two screenshots of the Flow Builder interface, showing the configuration of a 'Submit for Approval' action within a flow.

Top Screenshot: The flow diagram shows a 'Check Decision' decision element branching into 'Decision Approved' and 'Decision Rejected' paths. The 'Decision Approved' path leads to an 'Approve Claim' action, which then leads to an 'End' element. The 'Decision Rejected' path leads to the 'Submit for Approval' action configuration panel.

Bottom Screenshot: The 'Submit for Approval' configuration panel is shown with the following settings:

- Label:** Approve Claim
- API Name:** Approve_Claim
- Description:** (Empty text area)
- Set Input Values:**
 - Record ID:** A₃ recordid
 - Approval Process Name Or ID:** (Toggle set to 'Not Included')
- Skip Entry Criteria:** (Toggle set to 'Not Included')
- Submission Comments:** A₃ Claim Review Screen > Approver Comments (Toggle set to 'Included')
- Submitter ID:** (Toggle set to 'Not Included')
- View Output Resources:**
 - A₃ Instance ID
 - A₃ Instance Status
 - A₃ New Work Item IDs
 - A₃ Next Approver IDs
 - A₃ Record ID

- Add Action Element Under Decision Rejected Path
- Search for the submit for approval action and choose it
- Label it as Reject Claim
- Under Set Input Values
- Select variable recordID for Record ID
- Include the submission comments and Enter Approver Comments from Claim Review Screen



- Save and Activate the flow with Name Claim Approver Screen Flow

4.1 Create a Quick Action

- Go to setup, Click on Object Manager, Search for Claim Object and Select it
- In the left side panel, Click on Buttons, Links and Actions
- Click on Create New Action
- Select Action Type as Flow
- Select the flow that you have just created
- Label it as Approve/Reject Claim
- Name would be auto populated
- Click on Save

The screenshot shows the Salesforce Setup interface. The left sidebar has a menu with 'Buttons, Links, and Actions' selected. The main content area is titled 'Claim' and 'Approve/Reject Claim'. It displays the 'Action Detail' for the 'Approve/Reject Claim' action, which is a 'Flow' type. The action is associated with the 'Claim' object and the 'Claim Approver Screen Flow'.

Field	Value
Label	Approve/Reject Claim
Standard Label Type	
Name	Approve_Reject_Claim
Description	
Flow	Claim Approver Screen Flow
Created By	Insurance Portal, 12/5/2025, 2:48 AM
Modified By	Insurance Portal, 12/5/2025, 2:48 AM

- Add the quick action to the claim page layout
- In the left panel, Click on page layouts and open the claim page layout
- Go to mobile & lightning actions, Find the Approve/Reject Claim and Drag it to the Salesforce Mobile and Lightning Experience Actions
- Click on Save

The screenshot shows the Salesforce Setup interface. The left sidebar has a menu with 'Page Layouts' selected. The main content area is titled 'Claim' and 'Page Layouts'. It displays the 'Salesforce Mobile and Lightning Experience' actions for the 'Claim' object. The actions are listed in a table.

Field	Value
Approve/Reject Claim	
Post	
File	
New Event	
New Task	
New Contact	
Log a Call	
New Opportunity	
New Case	
New Lead	
Link	
Poll	
Question	
Email	
Change Owner	
Clone	
Printable View	
Delete	
Send Survey	
Sharing	
Edit Labels	
Edit	
Submit for Approval	
Change Record Type	

5. Write an extensive Apex Test Class for the ClaimsAdjusterController with 95%+ code coverage. Include test methods for complex claim filtering and data retrieval logic.

- Click on the setup icon, Right click on Developer Console and click on open link in new tab.
- On the top left corner, click on file, click on new, click on Apex class and give it a name ClaimsAdjusterControllerTest • Paste the following code:

```
@isTest
```

```
private class ClaimsAdjusterControllerTest {
```

```
    @testSetup
```

```
    static void setupTestData() {
```

```
        // Create a real active test user
```

```
        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
```

```
        User u = new User(
```

```
            FirstName = 'Test',
```

```
            LastName = 'User',
```

```
            Alias = 'tuser',
```

```
            Username = 'testuser1234343@example.com.test',
```

```
            Email = 'testuser123@example.com',
```

```
            TimeZoneSidKey = 'America/Los_Angeles',
```

```
            LocaleSidKey = 'en_US',
```

```
            EmailEncodingKey = 'UTF-8',
```

```
LanguageLocaleKey = 'en_US',
```

```
ProfileId = p.Id
```

```
);
```

```
insert u;
```

```
// Query Policy__c RecordType
```

```
Id policyRT =
```

```
Schema.SObjectType.Policy__c.getRecordTypeInfoByDeveloperName()
```

```
.get('Auto')
```

```
.getRecordTypeId();
```

```
Id claimRT =
```

```
Schema.SObjectType.Claim__c.getRecordTypeInfoByDeveloperName()
```

```
.get('Accident')
```

```
.getRecordTypeId();
```

```
// Create Contact owned by same user
```

```
Contact con = new Contact(
```

```
    FirstName = 'John',
```

```
    LastName = 'Doe',
```

```
    OwnerId = u.Id
```

```
);
```

```
insert con;
```

```
// Create Policy owned by same user
```

```
Policy__c policy = new Policy__c(
```

```
    Customer__c = con.Id,
```

```
    RecordTypeId = policyRT,
```

```
    OwnerId = u.Id,
```

```
    VIN__c = '1A903284375893532'
```

```
);
```

```
insert policy;
```

```
// Create Claim owned by same user
```

```
Claim__c claim = new Claim__c(
```

```
    Policy__c = policy.Id,
```

```
    RecordTypeId = claimRT,
```

```
    Claim_Amount__c = 5000,
```

```
    Date_of_Loss__c = Date.today().addDays(-10),
```

```
OwnerId = u.Id
```

```
);
```

```
insert claim;
```

```
System.debug('Claim inserted: '+claim.Id);
```

```
}
```

```
@isTest
```

```
static void testGetAssignedClaims() {
```

```
User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
```

```
System.runAs(u) {
```

```
Test.startTest();
```

```
List<ClaimsAdjusterController.ClaimWrapper> results =
```

```
ClaimsAdjusterController.getAssignedClaims();
```

```
System.debug('Result is: '+results);
```

```
Test.stopTest();
```

```
//System.assert(results.size() > 0,
```

```
//"Expected at least one wrapper returned");
```

```
ClaimsAdjusterController.ClaimWrapper wrap = results[0];
```

```
System.assertNotEquals(null, wrap.claimId);
```

```
System.assertNotEquals(null, wrap.claimNumber);
```

```
System.assertEquals('New', wrap.status);
```

```
System.assertEquals(5000, wrap.claimAmount);
```

```
System.assert(wrap.policyHolderName.contains('John'));
```

```
System.assertNotEquals('Unknown', wrap.policyType);
```

```
}
```

```
}
```

```
}
```

- Save the file using CTRL+S
- On the top right corner, you will find Run Test, Click on it to run the test class
- To check the test results, Down go for tests and you can see the how much percentage it covers your apex class

The screenshot shows the Salesforce Apex IDE interface. The top pane displays the source code for a test class named `ClaimsAdjusterControllerTest`. The code includes a `@isTest` annotation, a `private class` declaration, and a `@testSetup` method that creates a test user. The bottom pane shows the 'Tests' tab with a table of test run results. The table has columns for Status, Test Run, Enqueued Time, Duration, Failures, and Total. The 'Overall Code Coverage' table on the right shows that the `ClaimsAdjusterController` class has 86% code coverage (20/23 lines).

```

1 @isTest
2 private class ClaimsAdjusterControllerTest {
3
4     @testSetup
5     static void setupTestData() {
6
7         // Create a real active test user
8         Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
9         User u = new User(
10             FirstName = 'Test',
11             LastName = 'User',
12             Alias = 'tuser',
13             Username = 'testuser123434@example.com.test',
14             Email = 'testuser123@example.com',

```

Status	Test Run	Enqueued Time	Duration	Failures	Total
✖	TestRun @ 11:58:17 am			1	1
✖	TestRun @ 12:01:35 pm			1	1
✖	TestRun @ 12:07:16 pm			1	1
✖	TestRun @ 12:14:03 pm			1	1
✖	TestRun @ 12:22:36 pm			1	1
✖	TestRun @ 12:27:50 pm			1	1
✔	TestRun @ 12:29:39 pm			0	1

Class	Percent	Lines
Overall	48%	
ClaimsAdjusterController	86%	20/23
PremiumCalculator	0%	0/18

6. Refine the Sharing Rules to ensure Adjusters only see claims in their assigned territory/state.

- Go to setup, Search for flows in the quick find box and click on flows
- Click on New Flow
- Select Record Triggered Flow
- Select the object as claim
- Configure trigger when a record is created or updated
- Set entry conditions as none
- Optimize the flow for Updates

The screenshot shows the Salesforce Flow Builder interface. The flow is named 'Clame_policy_holder_state_update - V2'. The configuration panel on the right shows the 'Record-Triggered Flow' settings. The 'Object' is set to 'Claim'. The 'Trigger' is set to 'A record is created or updated'. The 'Optimize for: Fast Field Updates' option is selected. The 'Configure Trigger' section shows that the flow is triggered when a record is created or updated. The 'Set Entry Conditions' section shows that the entry conditions are set to 'None'.

Record-Triggered Flow
Start
Object: Claim
Trigger: A record is created or updated
Optimize for: Fast Field Updates
[Open Flow Trigger Explorer for Claim](#)

Configure Start
Object: Claim

Configure Trigger
Trigger the Flow When:
☐ A record is created
☐ A record is updated
☒ A record is created or updated
☐ A record is deleted

Set Entry Conditions
Specify entry conditions to reduce the number of records that trigger the flow and the number of times the flow is executed. Minimizing unnecessary flow executions helps to conserve your org's resources.
If you create a flow that's triggered when a record is updated, we recommend first defining entry conditions. Then select the **Only when a record is updated** to meet the condition requirements option for When to Run the Flow for Updated Records.
Condition Requirements: None

- Add get Records element into the flow canvas
- Label it as get_policy_records
- API name will be auto populated
- Set filter as None
- Store all the records
- Select Choose fields and let salesforce do the rest for How to store Record Data
- Under Select Policy fields to store fields in Variable, Select Policy State Field

The screenshot shows the Salesforce Flow Builder interface for a flow named 'Clame_policy_holder_state_update - V2'. The flow canvas displays the following elements in sequence:

- Record-Triggered Flow Start**: Object: Claim, Trigger: A record is created or updated, Optimize for: Fast Field Updates.
- get_policy_records**: Get Records element, currently highlighted.
- Claim_Filed_update**: Loop element, with a 'For Each' loop body and an 'After Last' exit point.

The configuration panel for the 'get_policy_records' element is open on the right, showing the following settings:

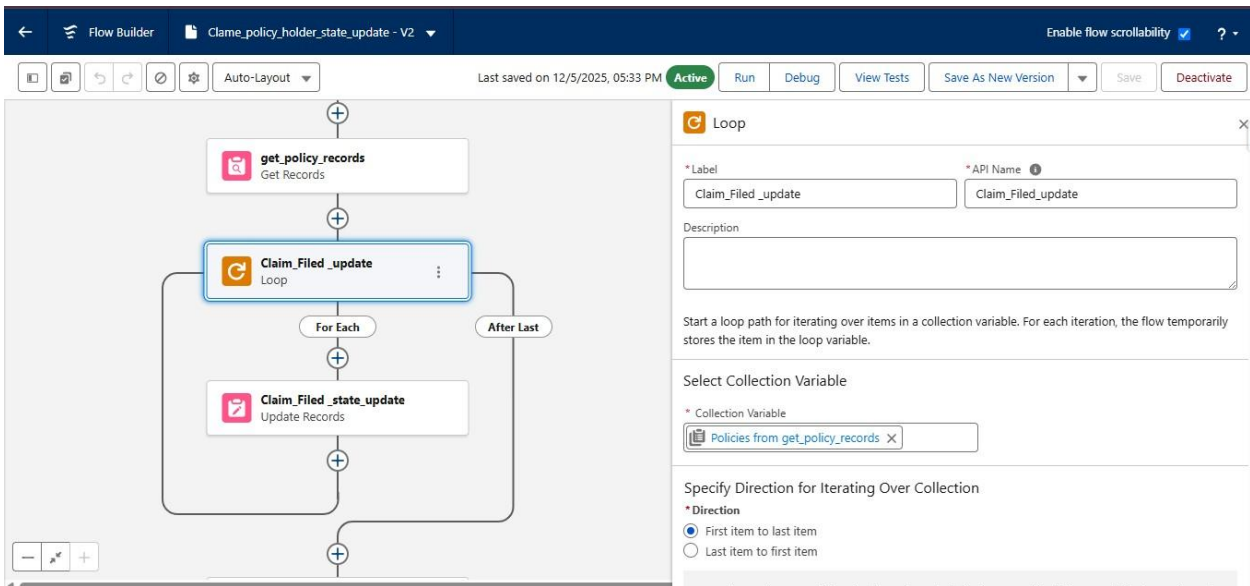
- Label**: get_policy_records
- API Name**: get_policy_records
- Object**: Policy
- Filter Policy Records**: Condition Requirements: None—Get All Policy Records.
- Sort Policy Records**: (Empty)

The screenshot shows the same Salesforce Flow Builder interface, but the configuration panel for the 'get_policy_records' element is now showing the 'How Many Records to Store' and 'How to Store Record Data' sections.

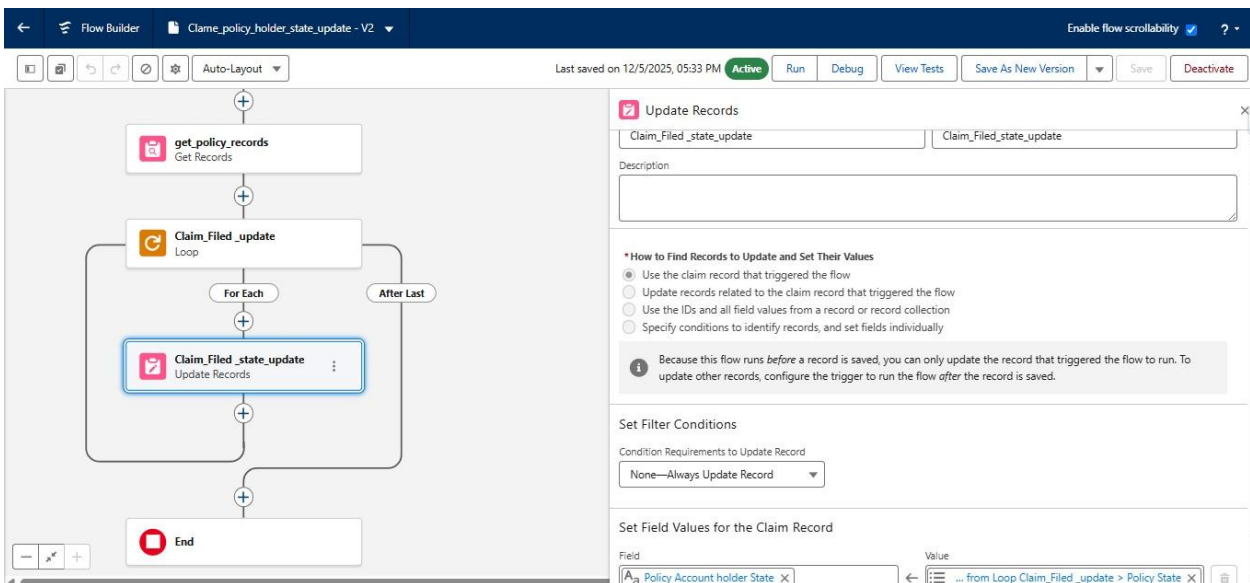
- How Many Records to Store**:
 - ☐ Only the first record
 - ☐ All records, up to a specified limit
 - ☒ All records
- How to Store Record Data**:
 - ☐ Automatically store all fields
 - ☒ Choose fields and let Salesforce do the rest
 - ☐ Choose fields and assign variables (advanced)
- Select Policy Fields to Store in Variable**:
 - Field: Id
 - Field: Policy_State__c
 - [+ Add Field](#)

- Add the Loop Element into the canvas after the get Records element
- Label it as Claim_Filed_update
- API name will be auto populated
- Under Select Collection Variables, Select policies from Get Policies from Get Policy Records

- Under Specify direction for Iterating over collection, Select first item to last item



- Under For each, Add the update records element into the canvas
- Label it as Claim_Filed_state_update
- API name will be auto populated
- Set filter conditions to NONE
- Under Set field values for the claim record, Select policy state from current loop and set it to the Policy Account Holder State field



- Save and Activate the flow with name Claim Policy Holder State Update

6.1 Define Sharing Rules for Claim Object

- Go to Set up, In the quick find box, Search for Sharing Settings
- Click on Sharing settings
- Under Manage Sharing Settings for, Select Claim Object
- Scroll down to find Claim Sharing Rules
- Click on new
- Label it as Share Records based on Policy State CA
- Rule Name will be Auto Populated
- Set the criteria to Policy Account Holder State Equals CA
- Give Access Level as Read/Write
- Click on Save

The screenshot shows the Salesforce Setup interface. In the left sidebar, the navigation menu includes 'Setup', 'Home', and 'Object Manager'. Under 'Security', there are links for 'Guest User', 'Sharing Rule Access Report', and 'Sharing Settings'. The main content area is titled 'Sharing Settings' and includes a warning: 'You can use sharing rules only to grant wider access to data, not to restrict access.' Below this, the configuration for a new sharing rule is shown. The 'Label' is 'Share Records based on Policy State CA' and the 'Rule Name' is 'Share_Records_based_on_Pol'. The 'Description' field is empty. Under 'Step 1: Select your rule type', there is a table with columns for 'Criteria', 'Field', 'Operator', 'Value', and a logical connector. The first row is configured with 'Policy Account holder State' as the field, 'equals' as the operator, and 'CA' as the value. Below this table, the 'Additional Options' section has a checked box for 'Include records owned by users who can't have an assigned role'. The 'Share with' group is 'Claim Adjusters- California' and the 'Access Level' is 'Read/Write'. The 'Created By' is 'Insurance Portal' on 12/7/2025 at 8:34 PM, and the 'Modified By' is also 'Insurance Portal' on the same date and time.

Criteria	Field	Operator	Value	
	Policy Account holder State	equals	CA	AND
	--None--	--None--		AND
	--None--	--None--		AND
	--None--	--None--		AND
	--None--	--None--		

Additional Options: ☒ Include records owned by users who can't have an assigned role

Share with: Group: Claim Adjusters- California

Access Level: Read/Write

Created By: Insurance Portal, 12/7/2025, 8:34 PM

Modified By: Insurance Portal, 12/7/2025, 8:34 PM

- Again Click on New
- Label it as Share Records based on Policy State
- Rule Name will be Auto Populated
- Set the criteria to Policy Account Holder State Equals TX
- Give Access Level as Read/Write
- Click on Save

Setup Home Object Manager

Q sharing

Security

Guest User Sharing Rule Access Report

Sharing Settings

Didn't find what you're looking for? Try using Global Search.

SETUP Sharing Settings

Label: Share Records based on Pol

Rule Name: Share_Records_based_on_P

Description:

Step 1: Select your rule type

Criteria	Field	Operator	Value	
	Policy Account holder State	equals	TX	AND
	--None--	--None--		AND
	--None--	--None--		AND
	--None--	--None--		AND
	--None--	--None--		AND

Add Filter Logic...

Additional Options

☐ Include records owned by users who can't have an assigned role

Share with: Group: Claim Adjusters - Texas

Access Level: Read/Write

Created By: Insurance Portal 12/7/2025, 8:33 PM

Modified By: Insurance Portal 12/7/2025, 8:33 PM

7. Finalize all Permission Sets to differentiate access between Agents (Quoting only), Adjusters (Claim handling), and Managers (Reporting/Approvals).

- Go to setup, In the quick find box, Search for Permission Sets, Click on Permission Sets
- Click on New
- Give the Label as Insurance Agent Access
- API Name will be Auto Populated

Setup Home Object Manager

Q permisio

Users

Permission Set Groups

Permission Sets

Custom Code

Custom Permissions

Didn't find what you're looking for? Try using Global Search.

SETUP Permission Sets

Save Cancel

Enter permission set information

Label: Insurance Agent Access

API Name: Insurance_Agent_Access

Description:

Session Activation Required ☐

Select the type of users who will use this permission set

Who will use this permission set?

-Choose "--None--" if you plan to assign this permission set to multiple users with different user and permission set licenses.

-Choose a specific user license if you want users with only one license type to use this permission set.

-Choose a specific permission set license if you want this permission set license auto-assigned with the permission set.

Not sure what a permission set license is? [Learn more here.](#)

License: --None--

Save Cancel

- Give the access for Objects
- Give Create, Read for Policy object
- Give Read, Edit to contact Object

- Give Read only to Claim object
- Make Flow User Enabled under System permissions
- Make PremiumCalculator Apex class enabled

7.1 Create Claims Manager Access Permission set

- Click on New
- Give the Label as Claims Manager Access
- API Name will be Auto Populated

The screenshot shows the Salesforce Setup interface for creating a new permission set. The left sidebar contains navigation links for 'Users', 'Permission Set Groups', 'Permission Sets', 'Custom Code', and 'Custom Permissions'. The main content area is titled 'SETUP Permission Sets' and includes a search bar. The 'Enter permission set information' section is highlighted, showing the 'Label' as 'Claims Manager Access' and the 'API Name' as 'Claims_Manager_Access'. The 'Session Activation Required' checkbox is unchecked. Below this, the 'Select the type of users who will use this permission set' section is visible, with a dropdown menu set to '--None--'.

- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to contact Object
- Give Read, Edit, Delete to Claim object
- Make Run Reports Enabled under System permissions
- Make Manage Approvals enabled under system permissions

7.2 Create Claims Adjuster Access Permission Set

- Click on New
- Give the Label as Claims Adjuster Access
- API Name will be Auto Populated

Search Setup

Setup Home Object Manager

permission

Users

Permission Set Groups

Permission Sets

Custom Code

Custom Permissions

Didn't find what you're looking for? Try using Global Search.

SETUP

Permission Sets

Save Cancel

Enter permission set information

Label Claims Adjuster Access

API Name Claims_Adjuster_Access

Description

Session Activation Required

Select the type of users who will use this permission set

Who will use this permission set?

-Choose '-None-' if you plan to assign this permission set to multiple users with different user and permission set licenses.
-Choose a specific user license if you want users with only one license type to use this permission set.
-Choose a specific permission set license if you want this permission set license auto-assigned with the permission set.

Not sure what a permission set license is? [Learn more here.](#)

License --None--

Save Cancel

- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to Claim object
- Make ClaimsAdjusterController Enabled under Apex Classes

Salesforce Project Link:

<https://orgfarm-021a70d1c8-dev-ed.develop.lightning.force.com/lightning/r/User/005bm00000Y07dpAAB/view>

